

OBJECT ORIENTED PROGRAMMING

INTERNAL WORKING OF JAVA

To understand how Java works internally, you have to look at its "Write Once, Run Anywhere" (WORA) philosophy. Unlike C++, where code is compiled directly into machine-specific code, Java uses an intermediate layer.

The internal workflow can be broken down into three main stages: **Compilation**, **Loading**, and **Execution**.

1. The Compilation Phase (Source to Bytecode)

When you write a .java file, you run the compiler (JDK)(javac).

- The compiler does not create a file your CPU can understand. Instead, it creates a **.class file** containing **Bytecode**.
- **Bytecode** is an optimized set of instructions designed to be executed by the **Java Virtual Machine (JVM)**. It is platform-independent.

2. The Loading Phase (The ClassLoader)

When you run the program (java MyClass), the JVM starts up and uses the **ClassLoader Subsystem** to bring the .class file into memory. It follows three steps:

1. **Loading:** Finds the .class file and imports the binary data.
2. **Linking:**
 - **Verify:** Ensures the bytecode is safe and doesn't violate Java security rules.
 - **Prepare:** Allocates memory for static variables and initializes them with default values.
 - **Resolve:** Replaces symbolic references with actual memory addresses.
3. **Initialization:** Executes static blocks and assigns actual values to static variables.

3. The Execution Engine (The Heart of JVM)

This is where the magic happens. The JVM converts Bytecode into Machine Code that your specific OS (Windows, Mac, Linux) can execute. It uses three main components:

A. The Interpreter

It reads bytecode instructions one by one and executes them. It is fast to start but slow for repeated code.

B. The JIT Compiler (Just-In-Time)

This is the secret to Java's speed. If the Interpreter finds a "Hot Spot" (code that is executed repeatedly, like a loop), the JIT compiler compiles that entire block into **Native Machine Code**.

- Next time that code runs, the JVM executes the native code directly, skipping the interpretation step.

C. The Garbage Collector (GC)

It runs in the background, identifying objects that are no longer reachable and reclaiming their memory so you don't have to manually manage memory.

4. Runtime Data Areas (JVM Memory)

While the code is running, the JVM organizes memory into five distinct areas:

1. **Method Area:** Stores class-level data (static variables, method code). Shared by all threads.
2. **Heap Area:** Where all **Objects** are stored. This is the primary target for Garbage Collection. Shared by all threads.
3. **Stack Area:** Created for every thread. It stores **Local Variables** and "Stack Frames" for method calls. Once a method finishes, its frame is "popped."
4. **PC Registers:** Contains the address of the current instruction being executed by the thread.
5. **Native Method Stack:** Stores instructions for code written in other languages (like C/C++ via JNI).

5. Summary: The Step-by-Step Flow

1. **You write:** Hello.java
2. **Javac compiles:** Hello.class (Bytecode).
3. **ClassLoader:** Loads the class into the **Method Area**.
4. **JVM Memory:** Allocates space in the **Stack** for the main method and **Heap** for objects.
5. **Execution Engine:** The **Interpreter** starts running code; the **JIT Compiler** optimizes it; the **GC** cleans up.

The Java Platform Independence (The "How" ?)

The most common opening question: *"Is Java truly platform-independent?"* The answer is: Java programs are platform-independent, but the JVM is platform-dependent.

- **Compilation Process:** Your .java file is compiled by javac into Bytecode (.class file). This bytecode is a highly optimized set of instructions that isn't specific to any processor.
- **Execution Process:** The Java Virtual Machine (JVM) reads this bytecode and converts it into machine code for the specific OS (Windows, Linux, Mac).

The Slogan: "Write Once, Run Anywhere" (WORA).

JDK vs. JRE vs. JVM (The Hierarchy)

Interviewers love to see if you know where the tools you use actually live.

Component	Full Name	Purpose	Contains
JVM	Java Virtual Machine	The engine that executes bytecode.	Interpreter, JIT Compiler, GC.
JRE	Java Runtime Environment	The environment needed to run Java apps.	JVM + Libraries (rt.jar).
JDK	Java Development Kit	The toolkit needed to develop Java apps.	JRE + Tools (javac, debugger).

Memory Management: Stack vs. Heap

This is where 70% of candidates stumble. Understanding where variables live is crucial for OOPS.

The Stack

- What lives here: Local variables, primitive types, and references to objects.
- Nature: Short-lived. Memory is allocated when a method is called and freed when it returns (LIFO).
- Scope: Each thread has its own Stack.

The Heap

- What lives here: All Objects (including strings and arrays).
- Nature: Long-lived. Managed by the Garbage Collector (GC).
- Scope: Shared across all threads in the application.

Scenario: If you write `String s = new String("Hi");`, the variable `s` (the reference) sits in the Stack, while the actual object "Hi" sits in the Heap.

The "Magic" of the JIT Compiler

Why is Java faster than a pure interpreter? The Just-In-Time (JIT) Compiler sits inside the JVM. It monitors which parts of your code are "hot" (run frequently). Instead of interpreting them line-by-line every time, it compiles those specific chunks directly into native machine code and caches them. This is why Java's performance improves as the application runs.

Potential Interview Questions (Basics)

1. Why is Java not 100% Object-Oriented?

Answer: Because of Primitive Data Types (int, char, boolean, etc.). They are not objects. However, with Wrapper classes and Autoboxing, we can treat them as such.

2. What is the difference between `main(String[] args)` and `static public void main(String args[])`?

Answer: Technically, none. The order of public static can be swapped, and the array brackets can be after the type or the variable name.

3. Why is the main method static?

Answer: So the JVM can call it without creating an instance of the class. If it weren't static, the JVM would face a "chicken and egg" problem: it needs to create an object to run the code, but it doesn't know how to run the code until the object is created.

4. Can we run a Java program without a main method?

Answer: Prior to Java 7, you could use a static block. In modern Java, you generally cannot, though you can run specific components like Applets or Servlets within a container.

Deep Dive Scenario: String Constant Pool (SCP)

In an interview, if they ask about memory, they will inevitably ask about Strings.

- `String s1 = "Java";` → Created in the String Constant Pool (a special area in the Heap).
- `String s2 = new String("Java");` → Forced into the General Heap.
- Why? To save memory. If 100 variables have the value "Java", they all point to the same reference in the SCP.

1. Data Types: The Subtle Traps

You know int, float, and boolean. But here is what an interviewer will actually ask:

- **Size Independence:** Unlike C/C++, the size of primitives in Java is **fixed** regardless of the platform (e.g., an int is always 32-bit). This is key to portability.
- **The char Mystery:** Why is a char 2 bytes (16-bit) in Java?
Answer: Java uses **Unicode** (UTF-16), not ASCII. This allows Java to support international languages.
- **Default Values:** Primitives have defaults (0, 0.0, false). **Crucial Point:** Local variables in a method **do not** get default values; they must be initialized before use, or the code won't compile. Only class-level members (fields) get defaults.

2. Wrapper Classes & Autoboxing

Java provides a "Wrapper" for every primitive (e.g., Integer for int).

- **Autoboxing:** Converting a primitive to a Wrapper automatically: Integer primitive = 5;
- **Unboxing:** Converting a Wrapper back to a primitive.
- **The Trap (Integer Caching):** `` ` ` java

Integer a = 127;

Integer b = 127;

System.out.println(a == b); // true (cached)

Integer c = 128;

Integer d = 128;

System.out.println(c == d); // false (not cached, new objects created)

> **Scenario:** Java caches Integers from -128 to 127. Outside this range, `==` compares memory addresses, not values. Always use `.equals()` for Wrappers!

3. The Java ClassLoader

Before a class can run, it must be loaded into memory. The JVM doesn't load all classes at once (to save memory); it loads them **on-demand**.

1. **Bootstrap ClassLoader:** Loads internal JDK classes (like java.lang.*).
2. **Platform/Extension ClassLoader:** Loads classes from the standard Java extension APIs.
3. **Application ClassLoader:** Loads classes from your CLASSPATH (the code you write).

Interview Question: "What is the Delegation Model?"

Answer: A ClassLoader always "delegates" the request to its parent first. It only tries to load the class itself if the parent can't find it. This prevents you from accidentally overwriting java.lang.String with your own version.

4. Access Modifiers (The Visibility Ladder)

This is the "Security" layer of OOPS.

Modifier	Class	Package	Subclass (Inheritance)	World
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes (via inheritance)	No
default	Yes	Yes	No	No
private	Yes	No	No	No

Pro Tip: If you don't specify a modifier, it is **default** (package-private). This is a common "trick" question.

5. Potential Interview Questions

1. What is the difference between == and .equals()?

Answer: == is an operator that compares memory addresses (references). .equals() is a method (originally from the Object class) designed to compare content/state.

2. Why are Wrapper classes immutable?

Answer: To ensure thread safety and so that they can be used in the Integer Cache and as keys in HashMaps without their value changing unexpectedly.

3. Can we make a top-level class private or protected?

Answer: No. A top-level class can only be public or default. Only inner (nested) classes can be private or protected.

The "Static" Logic

Before we hit the 4 Pillars of OOPS, we must master static.

- **Static Variable:** Shared by all instances of the class. It belongs to the class, not the object.
- **Static Method:** Can be called without an object. It **cannot** access non-static (instance) variables or methods directly.
- **Static Block:** Runs once when the class is first loaded into memory by the ClassLoader—even before the main method starts!

The Class: The Logical Template

A **Class** is a template or a set of instructions that defines the properties (attributes) and behaviors (methods) of a particular type of entity.

- **It occupies no space in the Heap memory.**
- It exists in the **Metaspace** (in Java 8+) or the **Method Area** (older versions), where the JVM stores the structure of the class, static variables, and method code.

Components of a Class:

1. **Fields (States):** Variables that define the data.
2. **Methods (Behaviors):** Functions that define what the object can do.
3. **Constructors:** Special blocks of code used to initialize the object.
4. **Nested Classes/Interfaces:** Classes within classes.

The Object: The Physical Reality

An **Object** is a physical entity that has state and behavior. It is an "instance" of a class.

- **Memory Location:** Every time you use the new keyword, a new object is created in the **Heap Memory**.
- **The Reference:** The variable you use to access the object (e.g., Dog myDog) is just a **pointer** residing in the **Stack Memory**.

```
public class Car {
    String color; // State

    void drive() { // Behavior
        System.out.println("The " + color + " car is moving.");
    }

    public static void main(String[] args) {
        Car c1 = new Car(); // Object 1 created
        c1.color = "Red";

        Car c2 = new Car(); // Object 2 created
        c2.color = "Blue";

        c1.drive(); // Logic: Which color is printed? Red.
    }
}
```

Real-World Scenario: The Architect and the Houses

Imagine an Architect (the Developer).

- **The Blueprint (Class):** The architect draws a plan for a "Villa." The plan specifies there must be 3 bedrooms, 1 kitchen, and a specific layout. The plan itself isn't a house; you can't live in a drawing.
- **The Villa (Object):** When a builder uses that blueprint to actually construct a house at *123 Maple Street*, that is an **Object**.
- **Multiple Instances:** The builder can use the *same* blueprint to build 50 identical villas in a neighborhood. Each villa has its own address (Memory Address) and its own color or furniture (State), even though they follow the same design.

What happens in the background?

1. **Loading:** The ClassLoader loads the Car.class into the Metaspace.
2. **Stack Entry:** c1 and c2 are created as reference variables in the **Stack**.
3. **Heap Entry:** new Car() allocates space in the **Heap** for two separate objects.
4. **Binding:** c1 holds the address of the "Red" object; c2 holds the address of the "Blue" object.

1. What is the difference between an Object and a Class?

Answer: A class is a logical entity/template; an object is a physical entity/instance. A class is loaded into the Metaspace once, while objects are created in the Heap as many times as needed.

2. Can you have a class without an object?

Answer: Yes. You can define a class and never instantiate it. You can also have a utility class with only static methods (like java.lang.Math), where objects are never needed.

3. What happens to an object when it is no longer referenced?

Answer: It becomes eligible for Garbage Collection. If c1 = null; is executed, the "Red" car object in the heap has no reference pointing to it, and the JVM will eventually reclaim that memory.

4. Is new the only way to create an object in Java?

Answer: No. You can create objects via Reflection (Class.forName().newInstance()), Cloning (clone()), Deserialization, or using the Factory Pattern.

Interviewers might ask: "Can an object exist without a name?" **Answer:** Yes! These are called **Anonymous Objects**. new Car().drive(); This creates an object on the heap, calls the method, and since there is no reference variable (like c1) to hold its address, it becomes eligible for Garbage Collection immediately after the line executes.

FOUR PILLARS OF OOPS

1.Encapsulation (The "Black Box")

Encapsulation is the practice of bundling data (variables) and the methods that act on that data into a single unit (a class) and restricting access to the inner workings.

How to achieve it:

- Declare variables as private.
- Provide public getter and setter methods.

Why do it? * **Data Validation:** You can prevent invalid data (e.g., setting age = -5) inside the setter.

- **Flexibility:** You can change the internal implementation without affecting the code that uses the class.
- **Read-only/Write-only:** You can skip the setter to make a field read-only.

2. Inheritance (The "Is-A" Relationship)

Inheritance allows one class (child/subclass) to acquire the properties and behaviors of another (parent/superclass). In Java, we use the `extends` keyword.

Crucial Inheritance Rules:

1. No Multiple Inheritance with Classes: A class cannot extend more than one class (to avoid the Diamond Problem). However, a class can implement multiple interfaces.
2. The `super` Keyword: `* super()` calls the parent's constructor.
`super.method()` calls the parent's version of an overridden method.
3. Constructor Execution: When you create a child object, the parent's constructor is executed first, then the child's.

3. The Diamond Problem (Common Interview Trap)

If Class A has a method `start()`, and Class B and C both override it, and Class D tried to inherit from both B and C—which `start()` would D use? This ambiguity is why Java disallows multiple inheritance of classes.

Note: Java 8 introduced "Default Methods" in interfaces, which brought a form of the Diamond Problem back. If a class implements two interfaces with the same default method, the compiler forces you to override and resolve the conflict manually.

4. Types of Relationships

Interviewers often ask the difference between Inheritance and Composition.

- IS-A (Inheritance): A Car is a Vehicle. Use `extends`.
- HAS-A (Composition/Aggregation): A Car has an Engine. You create an instance of Engine inside the Car class.
 - *Pro-tip:* In modern software design, "Composition over Inheritance" is often preferred because it's more flexible.

5. Potential Interview Questions

1. What is an "Immutable Class" and how do you create one?

Answer: A class whose state cannot be changed after creation (like `String`).

Steps: Make the class `final`, variables `private` and `final`, no setters, and perform deep copies for mutable objects in the constructor/getters.

2. Can a constructor be inherited?

Answer: No. Constructors are not members of a class. They are specific to the class they are defined in. They are *called* during the instantiation of a subclass, but not *inherited*.

3. What is the difference between `this()` and `super()`?

Answer: `this()` calls another constructor in the same class (constructor chaining). `super()` calls the constructor of the parent class. Both must be the first statement in a constructor.

1. Polymorphism (The "Flexibility" Pillar)

This is where Java gets its power. It's not just about methods with the same name; it's about how Java decides which code to run at runtime.

- Compile-time Polymorphism: Method Overloading.
- Runtime Polymorphism: Method Overriding and Dynamic Method Dispatch.
- Covariant Return Types: Can an overriding method return a different type than the parent? (Spoiler: Yes, but with rules).

2. Abstraction (The "Contract" Pillar)

- How do you design a system where you define *what* should be done but not *how*?
- Abstract Classes vs. Interfaces: This is the #1 most asked interview question.
- Java 8, 9, and 11 updates: Default methods, Static methods in interfaces, and Private methods in interfaces.
- Functional Interfaces & Lambda Expressions: How Java moved toward Functional Programming.

3. Exceptional Handling (The "Safety" Pillar)

Interviewer: "What happens if your code fails?" You need to know:

Checked vs. Unchecked Exceptions: Why does the compiler force you to handle some but not others?

The finally block vs. System.exit(): Does finally *always* run? (There's a trick answer here).

Try-with-resources: How modern Java prevents memory leaks.

4. The Collection Framework (The "Data" Pillar)

This is 40% of most technical rounds. You need to know the internal workings (Data Structures) of:

List: ArrayList (Dynamic Array) vs. LinkedList (Doubly Linked).

Set: How does a HashSet ensure uniqueness internally? (The hashCode() and equals() contract).

Map: The internal working of **HashMap** (Buckets, Collision handling, and the Java 8 Treeify threshold).

5. Multithreading & Concurrency (The "Performance" Pillar)

This separates the juniors from the seniors.

Lifecycle of a Thread: New, Runnable, Blocked, Waiting, Timed_Waiting, Terminated.

Synchronization: Locks, Volatile keywords, and Atomic variables.

Deadlocks: How to create one and, more importantly, how to fix one.

6. Java 8+ Features (The "Modern" Java)

If you don't know these, you look outdated.

Streams API: Processing data like a pipeline.

Optional Class: The end of the NullPointerException.

```
class Parent {
    static { System.out.println("Parent Static"); }
    { System.out.println("Parent Instance"); }
    Parent() { System.out.println("Parent Constructor"); }
}

class Child extends Parent {
    static { System.out.println("Child Static"); }
    { System.out.println("Child Instance"); }
    Child() { System.out.println("Child Constructor"); }
}
```

PILLARS IN REAL WORLD

1. Encapsulation (Data Hiding/Security)

The Concept: Bundling data and methods into a single unit and restricting access to the inner workings.

Real-World Scenario: An ATM Machine.

- The "Data": Your account balance, PIN, and transaction history.
- The "Restriction": You cannot manually change your balance by opening the ATM and writing a new number on the hard drive.
- The "Interface": You are provided with specific methods: withdraw(), deposit(), and checkBalance().
- The Benefit: The bank ensures that the balance only changes if you provide a valid PIN and have sufficient funds. This validates the data before updating it.

2. Abstraction (Hiding Complexity)

The Concept: Hiding the internal implementation details and showing only the essential features to the user.

Real-World Scenario: Sending a UPI Payment.

- The "Interface": You click a "Pay" button and enter a PIN.
- The "Implementation": You don't see the complex handshake between your bank's server, the NPCI server, and the receiver's bank. You don't know if they are using SQL or NoSQL, or if the server is running on Linux or Windows.
- The Benefit: You only care about the contract ("If I provide a PIN and an amount, the money moves"). This reduces the complexity for the user.

3. Inheritance (Reusability/Hierarchy)

The Concept: A mechanism where one class acquires the properties and behaviors of a parent class.

Real-World Scenario: Different Types of Bank Accounts.

- The Parent (Base Class): Account. It has common features like accountNumber, name, and calculateInterest().
- The Children (Subclasses): SavingsAccount and CurrentAccount.
- The "Is-A" Relationship: A SavingsAccount is an Account.
- The Benefit: You don't have to rewrite the accountNumber logic for every new account type. You "inherit" the common code and only add specific features (like minimumBalance for Savings).

4. Polymorphism (Flexibility/Many Forms)

The Concept: The ability of a single function or object to behave differently in different contexts.

A. Compile-time (Overloading)

Scenario: The openAccount() method.

- openAccount(String name, String PAN) — Open with basic details.
- openAccount(String name, String PAN, double initialDeposit) — Open with an initial balance.

Logic: The same method name behaves differently based on what you provide.

B. Runtime (Overriding)

Scenario: The calculateInterest() method.

- The Account (Parent) has a default interest of 2%.
- The SavingsAccount overrides it to 4%.
- The FixedDepositAccount overrides it to 7%.

The "Magic": If a Bank Manager (the JVM) looks at a list of generic Account objects and calls calculateInterest(), the system automatically picks the correct rate based on the actual type of account at that moment.

Summary for the Interviewer

Pillar	Key Keyword	One-Sentence Pitch
Encapsulation	Security	"I hide my data and control access through methods."
Abstraction	Design	"I hide the 'how' and only show the 'what'."
Inheritance	Reusability	"I don't reinvent the wheel; I build upon existing code."
Polymorphism	Flexibility	"I allow one interface to handle multiple underlying forms."

The Collection Framework.

The Collection Framework is a unified architecture for storing and manipulating a group of objects.

- **List:** Ordered collection, allows duplicates (e.g., ArrayList, LinkedList).
- **Set:** Unordered collection, **prohibits** duplicates (e.g., HashSet, LinkedHashSet, TreeSet).
- **Map:** Stores Key-Value pairs. Not a true "Collection" but part of the framework (e.g., HashMap, TreeMap).

HashSet: How it ensures Uniqueness

This is a high-frequency question. How does a HashSet know if an object is a duplicate?

1. When you add an object, Java calls its hashCode().
2. The hash code determines the "bucket" location.
3. If the bucket is empty, the object is added.
4. If there's already an object, Java calls .equals().
 5. If .equals() returns true, the duplicate is rejected.
 6. If false, it's stored via a linked list/tree (Collision).

Feature	ArrayList	LinkedList
Internal Structure	Resizable Array.	Doubly Linked List.
Access (Search)	$O(1)$ (Random access via index).	$O(n)$ (Must traverse from head).
Manipulation (Add/Remove)	$O(n)$ (Requires shifting elements).	$O(1)$ (Just change pointers).
Best Use Case	When you need to search frequently.	When you need to add/remove frequently.

Internal Working of HashSet

Here is the secret: **HashSet doesn't have its own storage logic.** When you create a HashSet, it initializes a HashMap behind the scenes.

How it works:

1. When you do HashSet.add("Apple"), it actually calls map.put("Apple", PRESENT).
2. The **Key** is your object ("Apple"), and the **Value** is a dummy constant object called PRESENT.
3. Since a HashMap cannot have duplicate keys, the HashSet naturally ensures uniqueness.

```
// Inside the HashSet source code:  
private transient HashMap<E, Object> map;  
private static final Object PRESENT = new Object(); // Dummy value  
  
public boolean add(E e) {  
    return map.put(e, PRESENT) == null;  
}
```

Internal Working of HashMap (The "Gold" Question)

An interviewer will almost certainly ask: "How does a HashMap work internally in Java 8?"

- **Initial Capacity:** Default is 16 buckets.
- **Hashing:** The key's hashCode() is used to find the bucket index.
- **Collision Handling:** If two keys land in the same bucket, they are stored in a Linked List.
- **Java 8 Optimization (Treeification):** If a single bucket exceeds **8 elements** (and total capacity > 64), the Linked List is converted into a **Balanced Red-Black Tree**.

Why? To improve search performance from **O(n)** to **O(log n)**.

A HashMap works on the principle of **Hashing**. It uses an array of "Buckets" to store key-value pairs.

The Components:

- **Node<K,V>:** Each entry in the map is a Node containing: int hash, K key, V value, and Node next.
- **Threshold:** When the map gets too full (Load Factor, usually 0.75), it doubles in size (Resizing).

The Example Scenario:

Imagine we want to store:

1. {"Apple": 10}
2. {"Banana": 20}
3. {"Cherry": 30}

The Step-by-Step Process:

1. Calculate Hash: When you call `map.put("Apple", 10)`, the JVM calls `"Apple".hashCode()`. Let's say it returns 12345.
2. Calculate Index: The JVM uses a formula: $\text{index} = \text{hash} \& (n-1)$ (where n is array length). Suppose the index is 4.
3. Store the Node: A Node is created and placed at index 4.
4. Handle Collision: What if "Banana" also results in index 4?
 - Before Java 8: It was added as a Linked List at that index.
 - After Java 8: If the number of items in one bucket exceeds 8 (the `TREEIFY_THRESHOLD`), the Linked List converts into a Red-Black Tree. This improves search time from $O(n)$ to $O(\log n)$.

What is the difference between Fail-Fast and Fail-Safe iterators?

- **Answer: Fail-Fast** (`ArrayList`, `HashMap`) throws `ConcurrentModificationException` if the collection is modified while iterating. **Fail-Safe** (`CopyOnWriteArrayList`) works on a clone and doesn't throw exceptions.

Why should a Key in a HashMap be Immutable?

- **Answer:** If the key's state changes, its `hashCode()` changes. You would never be able to find that object again in the Map because it would be looking in the wrong bucket. This is why `String` and `Integer` are the most popular keys.

Difference between Set and List?

- **Answer:** List is an ordered sequence that allows duplicates. Set is an unordered collection that ensures uniqueness.

The "hashCode() and equals()" Contract

This is the most common interview trap. If you store a custom object (like a `Student` class) in a Map/Set, you **must** override these two methods.

The Scenario:

You have two objects: `Student s1 = new Student(101)` and `Student s2 = new Student(101)`.

- **If you don't override hashCode():** They will generate different hashes, go to different buckets, and you will have "duplicates" in your Set.
- **If you override hashCode() but not equals():** They will go to the same bucket. The JVM will see a collision, call `s1.equals(s2)`, which returns false (default behavior compares memory addresses), and still add both.

Gold Rule: > 1. If `obj1.equals(obj2)` is true, then `obj1.hashCode() == obj2.hashCode()` **must** be true.

2. If `hashCode()` is the same, the objects are **not necessarily** equal (this is a collision).

4. Possible Interview Questions

Q: What is the time complexity of get() and put()?

Answer: Ideally $O(1)$. In the worst case (many collisions), it's $O(\log n)$ due to the Red-Black tree.

Q: What happens if the hashCode() of all keys is the same?

Answer: All entries will go into a single bucket. The Map will effectively turn into a Balanced Tree (or a Linked List in older Java versions), degrading performance.

Q: Can we store null in HashMap?

Answer: Yes, **one** null key (stored at index 0) and multiple null values. `Hashtable` (the older version) does not allow nulls.

Exception Handling

An Exception is an **unwanted or unexpected event** that occurs during the execution of a program (at runtime) which disrupts the normal flow of instructions.

Interview Point: In Java, every Exception is an **Object**. When an error occurs, the JVM creates an object of the corresponding Exception class (like `ArithmeticException`) and "throws" it.

The Hierarchy (The "Family Tree")

You must be able to visualize this hierarchy. All exception classes are subtypes of the `java.lang.Throwable` class.

- **Throwable:** The parent of all errors and exceptions.

- **Error:** Indicated by java.lang.Error. These are "Fatal" (e.g., StackOverflowError, OutOfMemoryError). An application **should not** try to catch these because you can't recover from a dead JVM.
- **Exception:** Indicated by java.lang.Exception. These are conditions your code **should** handle.

The Two Pillars: Checked vs. Unchecked

A. Checked Exceptions (Compile-time Exceptions)

- Definition: Exceptions that are checked by the compiler at compile-time.
- The Rule: You are forced to either handle them (try-catch) or declare them (throws).
- Examples: IOException, SQLException, ClassNotFoundException.
- Real-world scenario: You are trying to read a file from a hard drive. The compiler says: *"What if the file is missing? You MUST write a plan for that before I compile this."*

B. Unchecked Exceptions (Runtime Exceptions)

- Definition: Exceptions that occur at runtime and are NOT checked by the compiler.
- The Rule: These usually happen due to poor logic.
- Examples: NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException.
- Real-world scenario: You try to divide a number by a variable that happens to be zero. The compiler doesn't know what the variable will be at runtime, so it doesn't complain during compilation.

Keyword	Explanation
try	Defines a block of code where we "expect" an exception. It must be followed by catch or finally.
catch	The "Handler." This block executes only if an exception occurs in the try block.
finally	The "Cleanup" block. It executes no matter what (even if there is a return in try/catch).
throw	Used to explicitly throw an exception (e.g., if(age<18) throw new InvalidAgeException();).
throws	Used in the method signature to "pass the buck" (delegate) the responsibility of handling to the caller.

Q1: Can we have a try block without a catch?

Answer: Yes, but only if you have a finally block. A try block must be followed by at least one of them.

Q2: What is "Try-with-Resources" (Java 7+)?

Answer: It's a try block that declares one or more resources (like a File or DB connection). It ensures that the resource is **automatically closed** at the end of the statement. The resource must implement the AutoCloseable interface.

Q3: What is Exception Propagation?

Answer: If an exception is not caught in the current method, it "drops" down to the calling method in the Stack. This continues until it is caught or reaches the JVM, which then terminates the program. **Note:** Only unchecked exceptions propagate by default; checked exceptions must be declared using throws to propagate.

Q4: The finally vs System.exit(0) trick.

Interviewer: *"Is there any case where the finally block does not execute?"* **Answer:** Yes. If System.exit(0) is called in the try or catch block, or if the JVM crashes/the power goes out, finally will not run.

Real-World Example: Banking Transaction

Imagine you are writing code for a Money Transfer:

1. **try:** Start the transaction, deduct money from Account A.
2. **catch (InsufficientFundsException e):** Stop the process and alert the user (Custom Exception).
3. **catch (SQLException e):** If the database is down, log the error for the tech team.
4. **finally: Close the database connection.** This is critical. If you don't close it, your server will eventually run out of connections and crash (Resource Leak).

Java

```

try {
    int a = 10 / 0;
} catch (Exception e) {
    System.out.println("General Exception");
} catch (ArithmeticException e) {
    System.out.println("Arithmetic Exception");
}

```

- **The Problem:** This will cause a **Compile-time Error**.
- **Why:** `ArithmeticException` is a subclass of `Exception`. Java checks catch blocks in order. Since the first block catches *all* exceptions, the second block is "unreachable code."
- **The Fix:** Always catch the **Subclass** first, then the **Superclass**.

Java

```

public int testMethod() {
    try {
        return 10;
    } catch (Exception e) {
        return 20;
    } finally {
        return 30;
    }
}

```

- **Question:** What does this return?
- **Answer:** 30.
- **Why:** In Java, the `finally` block is guaranteed to execute. If it contains a `return` statement, it overrides any `return` value from the `try` or `catch` blocks. The JVM literally "pops" the previous return value off the stack and replaces it with the one from `finally`.

The Scenario:

1. **Method C** throws a `NullPointerException`.
2. **Method C** doesn't have a try-catch.
3. The Exception "drops" to **Method B** (which called C).
4. **Method B** doesn't handle it; it drops to **Method A** (which called B).
5. If **Method A** has a try-catch, the exception is finally handled there.

Interview Question: "Does this work for Checked Exceptions?" **Answer:** No. For **Checked Exceptions**, each method in the chain must either catch it or explicitly say throws `IOException` in its signature.

Aspect	Explanation
WHAT	An object describing an error condition.
WHY	To prevent the program from crashing (Graceful Termination).
WHERE	Anywhere in the code where external factors (I/O, DB) or logic errors occur.
HOW	By using try to monitor, catch to handle, and finally to clean up.

Catch Specific Exceptions: Don't just catch the generic `Exception` class. Catch the specific one (like `FileNotFoundException`) so you know exactly what went wrong.

Don't Swallow Exceptions: Never leave a catch block empty. If you catch an error, at least log it so you can debug it later.

Cleanup in Finally: Always use `finally` to close resources (files, sockets, or streams) to prevent memory leaks.

Custom Exceptions: In large OOP projects, you can create your own exception classes by extending the `Exception` class to represent business-specific errors (e.g., `InsufficientFundsException`).

CREATING CUSTOM EXCEPTIONS

1. The Basic Formula

To create a custom exception, you simply create a class that **extends** one of Java's built-in exception classes:

1. **Extend Exception:** If you want a **Checked Exception** (the compiler forces the caller to handle it).
2. **Extend RuntimeException:** If you want an **Unchecked Exception** (cleaner code, but less safety enforcement).

```

// 1. Create the class extending Exception
public class AgeInvalidException extends Exception {

    // 2. Call the parent constructor with a custom message
    public AgeInvalidException(String message) {
        super(message);
    }
}

```

Step 1: Define the Custom Exception

```

public class InsufficientFundsException extends Exception {
    private double amountMissing;

    public InsufficientFundsException(double amountMissing) {
        super("Sorry, you are short by: $" + amountMissing);
        this.amountMissing = amountMissing;
    }

    public double getAmountMissing() {
        return amountMissing;
    }
}

```

Step 2: Throw the Exception in a Method

You use the throw keyword to trigger the exception and throws in the method signature to warn the caller.

```

public class BankAccount {
    private double balance = 100.00;

    public void withdraw(double amount) throws InsufficientFundsException {
        if (amount > balance) {
            double missing = amount - balance;
            // Triggering our custom exception
            throw new InsufficientFundsException(missing);
        }
        balance -= amount;
        System.out.println("Withdrawal successful!");
    }
}

```

Step 3: Catch the Custom Exception

```

public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        try {
            account.withdraw(500.00);
        } catch (InsufficientFundsException e) {
            System.out.println("Error: " + e.getMessage());
            System.out.println("Please deposit at least $" + e.getAmountMissing());
        }
    }
}

```

Why go through the trouble?

Using custom exceptions provides several benefits:

- **Business Logic Clarity:** Instead of catching a generic RuntimeException, you catch an InsufficientFundsException. It tells a story.
- **Extra Data:** As shown in the example above, you can add fields (like amountMissing) to the exception class to pass helpful data back to the user or logger.
- **Separation of Concerns:** It keeps your "happy path" code (the math) separate from your "error path" code (the alerts).

1. Lambda Expressions (Functional Interfaces)

Before Java 8, if you wanted to pass a piece of logic to a method, you had to create a whole class or an anonymous inner class.

What: A Lambda is an anonymous function (a function with no name, no return type, and no modifiers).

Why: it makes the code concise and allows you to treat "functionality" as a method argument.

Syntax: (parameters) -> { body }

Real-World Scenario: Sorting a list of Students by their marks.

Old way: Create a Comparator class, override compare(), instantiate it.

Lambda way: list.sort((s1, s2) -> s1.getMarks() - s2.getMarks());

2. Stream API (Data Processing)

The Stream API is **NOT** about I/O streams (like reading files). It is about processing collections of objects in a **declarative** way (telling the computer *what* you want, not *how* to loop).

The Pipeline Structure:

1. **Source:** (e.g., List, Set, Array)
2. **Intermediate Operations:** (Transform the stream)
 - filter(): Selects elements based on a condition.
 - map(): Converts one object into another (e.g., Student object to String name).
 - sorted(): Sorts the elements.
3. **Terminal Operation:** (Produces a result or side-effect)

collect(): Converts the stream back into a List/Set.
forEach(): Performs an action for each element.
count(): Returns the number of elements.

3. Optional Class (Ending the "Null" Nightmare)

The NullPointerException (NPE) is the most famous error in Java history. Java 8 introduced Optional<T> to handle cases where a value might be missing without using null.

- **The Problem:** String name = student.getName(); // If student is null, CRASH!
- **The Solution:** Optional<String> name = Optional.ofNullable(student.getName()); You can then use .orElse("Unknown") to provide a default value gracefully.

4. Default and Static Methods in Interfaces

As we touched on in OOPS, Java 8 broke the rule that interfaces can only have abstract methods.

- **Default Methods:** Use the default keyword. They allow you to add new methods to interfaces without breaking the classes that implement them.
- **Static Methods:** Use the static keyword. They work just like utility methods inside the interface.

What is a Functional Interface?

Answer: An interface that has exactly one abstract method (e.g., Runnable, Callable, Comparator). You can use the @FunctionalInterface annotation to ensure this.

Difference between map() and flatMap()?

Answer: map() is used for simple transformation (1-to-1). flatMap() is used when each element of a stream creates its own stream (1-to-many), and you want to "flatten" them into a single stream.

Are Streams Lazy or Eager?

Answer: Lazy. Intermediate operations are not executed until a terminal operation is called. This is why Streams are so efficient.

What is Garbage Collection?

Garbage Collection is the process by which the JVM identifies and deletes **unreachable objects** from the **Heap Memory** to reclaim space for new objects.

An object is eligible for GC if:

- It has no references pointing to it (it's "nullified").
- It was created inside a method that has finished execution (local variable scope ended).
- It is part of an "Island of Isolation" (two objects point to each other but nothing else points to them).

2. The How: "Mark and Sweep" Algorithm

Most GC implementations follow these two core phases:

1. **Marking:** The GC traverses the object graph starting from "GC Roots" (Stack references, static variables). It marks every object it can reach as **Alive**.
2. **Sweeping:** The GC scans the Heap and identifies objects that weren't marked. These are "Garbage," and their memory is cleared.
3. **Compacting (Optional):** After sweeping, the GC moves the remaining live objects together to eliminate gaps (fragmentation) in memory.

3. The Where: Generational Heap Memory

The JVM divides the Heap into generations based on the "**Weak Generational Hypothesis**"—the idea that most objects die almost immediately after creation.

A. Young Generation

Where all new objects are born.

- **Eden Space:** Most new objects start here.
- **Survivor Spaces (S0 and S1):** Objects that survive a GC cycle in Eden move here.
- **Minor GC:** This is a frequent, fast GC that happens in the Young Generation.

B. Old (Tenured) Generation

If an object survives enough rounds of Minor GC (the "threshold"), it is promoted (tenured) to the Old Generation.

- **Major GC / Full GC:** This cleans the Old Generation. It is much slower and triggers a "**Stop the World**" event.

4. "Stop the World" (STW)

This is a critical interview term. During GC, the JVM must freeze all application threads to ensure that the object graph doesn't change while the GC is trying to map it.

- **Goal of Modern GC:** Minimize the duration of STW so the user doesn't notice a "lag" in the application.

5. Types of Garbage Collectors

The interviewer might ask, "Which GC should I use for a low-latency app?"

Collector	Characteristics	Best Use Case
Serial GC	Uses a single thread for GC. Freezes everything.	Simple, small apps with tiny heaps.
Parallel GC	Uses multiple threads for the Young Gen.	Throughput-heavy apps (Batch processing).
G1 (Garbage First)	Divides heap into regions; cleans regions with the most garbage first.	Large heaps; balances throughput and latency (Default since Java 9).
ZGC / Shenandoah	Experimental/Ultra-low latency; performs most work concurrently.	Apps requiring sub-millisecond pause times.

6. Interview "Trick" Questions on GC

Q1: Can we force Garbage Collection in Java?

- **Answer:** No. You can call `System.gc()` or `Runtime.getRuntime().gc()`, but these are just **hints** or "suggestions" to the JVM. The JVM decides when to actually run the GC based on memory pressure.

Q2: What is the `finalize()` method?

- **Answer:** It is a method in the `Object` class that is called by the GC just before an object is destroyed.
- **Interview Tip:** Mention that `finalize()` is **deprecated** and unreliable. Use `try-with-resources` or `finally` for cleanup instead.

Q3: Does Garbage Collection prevent all Memory Leaks?

- **Answer:** No. A memory leak occurs when an object is no longer needed but remains **reachable** (e.g., adding objects to a static `HashMap` and never removing them). GC cannot delete these because it thinks they are still in use.

Q4: What is a "Memory Leak" vs. "Memory Fragmentation"?

- **Answer:** A leak is unreachable-but-held memory. Fragmentation is when there are many small gaps in memory that are too small for a new large object, even if the total free space is sufficient.

WORKING OF GC

Step 1: Birth in Eden

Every time you write `new MyObject()`, it is allocated in the **Eden Space** (part of the Young Generation).

Step 2: The First "Minor GC"

When Eden gets full, a **Minor GC** is triggered.

1. The JVM performs **Marking**: It starts from "GC Roots" (thread stacks, static variables) and identifies which objects are still in use.
2. Unreachable objects are deleted.
3. **The Survivors**: Objects that are still alive are moved to the first **Survivor Space (S0)**. Their "age" (tenure) is set to 1.

Step 3: Survival of the Fittest

The next time Eden fills up, another Minor GC occurs:

1. It checks **Eden AND S0**.
2. Surviving objects from both areas are moved to **S1**.
3. S0 is cleared out completely.
4. The survivors' age is incremented.

Note: S0 and S1 act like a "ping-pong" match. One is always empty, serving as a clean landing zone for the next GC cycle.

Step 4: Promotion (Tenuring)

When an object's age reaches a certain threshold (default is **15**), it is "promoted." It moves from the Young Generation to the **Old Generation (Tenured Space)**.

2. Major GC vs. Full GC

- **Major GC:** Cleans the **Old Generation**. It happens less frequently but takes much longer because the Old Gen is much larger than the Young Gen.
- **Full GC:** This is the "Nuclear Option." It cleans the **entire Heap** (Young + Old + Metaspace). This causes the longest **Stop-the-World** pauses.

3. The "How": Mark-Sweep-Compact Algorithm

This is the logic the Execution Engine uses:

1. **Mark:** The collector identifies which objects are still "reachable."
2. **Sweep:** It removes the unreachable ones, leaving "holes" in memory.
3. **Compact:** To prevent **Memory Fragmentation**, it slides all the surviving objects to the start of the heap. This ensures that the remaining free space is one big, contiguous block, making future allocations much faster.

4. Advanced Interview Questions (The "Internal" Level)

Q1: What are "GC Roots"?

Answer: To know what is "garbage," the JVM needs a starting point. GC Roots are objects that are always reachable. They include:

- Local variables in the currently executing methods (Stack).
- Active Java Threads.
- Static variables of loaded classes.
- JNI (Native) references.

Q2: What is a "Safe Point"?

Answer: To perform a "Stop-the-World" GC, the JVM cannot just freeze a thread mid-calculation. It waits for all threads to reach a **Safe Point** (like a loop jump or a method return) where the thread's state is consistent and it can be safely paused.

Q3: Why is the Young Generation split into Eden and Survivor spaces?

Answer: To prevent **fragmentation**. By copying survivors from Eden to S1 and clearing Eden entirely, the JVM keeps the memory "compact" without having to do a heavy compaction algorithm every few seconds. Copying is faster than sweeping "holes."

Q4: What is the TLAB (Thread Local Allocation Buffer)?

Answer: (Expert Level) To avoid "locking" the Heap every time a thread creates an object, each thread is given a small private slice of the Eden space called a **TLAB**. It can create objects there without talking to other threads. This makes object creation in Java nearly as fast as in C++.

5. Summary Table for Quick Revision

Feature	Minor GC	Major/Full GC
Target	Young Gen (Eden + S0/S1)	Old Gen (and sometimes Metaspace)
Frequency	High (Very common)	Low (Occasional)
Speed	Very Fast (milliseconds)	Slow (can be seconds)
Pause Type	Stop-the-World	Stop-the-World

Java Design Patterns.

In a placement interview, once you prove you can code, the interviewer will ask: *"How do you structure your code to make it maintainable and scalable?"* They are looking for your knowledge of **Design Patterns**—proven solutions to recurring software design problems.

1. The Singleton Pattern (Creational)

The Scenario: You need a single point of access to a resource, like a Database Connection Pool or a Configuration Manager.

- **The Goal:** Ensure a class has only **one instance** and provide a global point of access to it.
- **Interview Trap:** "How do you make it thread-safe?"
- **Expert Solution:** Use a private static instance, a private constructor, and a public static method with **Double-Checked Locking**.

Java

```
public class DatabaseConnection {
    private static volatile DatabaseConnection instance; // volatile for thread visibility
    private DatabaseConnection() {} // private constructor

    public static DatabaseConnection getInstance() {
        if (instance == null) { // First check
            synchronized (DatabaseConnection.class) {
                if (instance == null) { // Second check
                    instance = new DatabaseConnection();
                }
            }
        }
    }
}
```

```

    return instance;
}
}

```

2. The Factory Pattern (Creational)

The Scenario: You are building a Notification System. The user can choose to send an Email, SMS, or Push Notification.

- **The Goal:** Define an interface for creating an object, but let subclasses decide which class to instantiate.
- **Why:** It keeps your code **decoupled**. If you add a "WhatsApp" notification later, you only change the Factory, not the main logic.

3. The Observer Pattern (Behavioral)

The Scenario: Think of a YouTube Channel or a Stock Market app.

- **The Goal:** When one object (the Subject) changes state, all its dependents (Observers) are notified automatically.
- **Real-world:** When a "Price" object changes in a trading app, the "UI Graph," the "Alert System," and the "Log System" all update simultaneously.

4. The Strategy Pattern (Behavioral)

The Scenario: A Shopping Cart where you can pay via Credit Card, PayPal, or Crypto.

- **The Goal:** Define a family of algorithms, encapsulate each one, and make them interchangeable.
- **OOPS Logic:** This heavily uses **Polymorphism**. You pass the "Strategy" (Payment Method) into the "Context" (Checkout).

5. Potential Interview Questions

1. **What is the difference between a Singleton and a Static Class?**
 - *Answer:* A Singleton is an object (stored on the Heap) and can implement interfaces or extend classes. A static class is just a collection of methods. Singletons allow for **lazy initialization** (created only when needed), whereas static members are loaded when the class is first referenced.
2. **Explain "Dependency Injection" (DI).**
 - *Answer:* DI is a design pattern where an object's dependencies are provided externally rather than the object creating them itself. It promotes **loose coupling**.
3. **When would you use the Adapter Pattern?**
 - *Answer:* When you have two incompatible interfaces that need to work together (like a 3-pin plug in a 2-pin socket). You create an "Adapter" class that wraps the old interface to look like the new one.

6. The "SOLID" Principles (The Holy Grail)

Interviewers often ask: "What makes a design 'good'?" You respond with **SOLID**:

Letter	Principle	Summary
S	Single Responsibility	A class should have only one reason to change.
O	Open/Closed	Classes should be open for extension but closed for modification.
L	Liskov Substitution	Subtypes must be substitutable for their base types.
I	Interface Segregation	Don't force a class to implement methods it doesn't use.
D	Dependency Inversion	Depend on abstractions, not on concrete implementations.

Final Scenario for You

Question: "You are designing an app like Uber. You have different pricing logic for 'Rainy Days', 'Peak Hours', and 'Normal Days'. Which design pattern would you use to manage these different calculation logics?"

The Answer: Strategy Pattern. You define a PricingStrategy interface and implement RainyDayStrategy, PeakHourStrategy, etc. At runtime, the app injects the correct strategy based on the weather or time.

Interface

1. What is an Interface?

An interface is a reference type in Java, similar to a class, but it can **only** contain constants, method signatures, default methods, static methods, and nested types.

- **Implicit Modifiers:**
 - All variables are implicitly public static final.
 - All methods (unless specified as default/static) are implicitly public abstract.

2. Why use Interfaces?

Interviewers look for these three key reasons:

1. **Total Abstraction:** It hides the implementation details entirely.

2. **Multiple Inheritance:** A class cannot extend two classes, but it **can** implement multiple interfaces. This solves the "Diamond Problem."
3. **Loose Coupling:** It allows you to swap implementations without changing the calling code. (e.g., swapping a MySQL database for MongoDB if they both implement a Database interface).

3. The Evolution (Java 8 and 9)

This is where 90% of candidates get tested. Interfaces are no longer "purely" abstract.

A. Default Methods (Java 8)

Allows you to add new methods to interfaces without breaking the existing classes that implement them.

Java

```
interface Vehicle {
    void start();
    default void honk() {
        System.out.println("Beep beep!");
    }
}
```

B. Static Methods (Java 8)

Used for utility methods that belong to the interface class, not the object. They cannot be overridden.

C. Private Methods (Java 9)

Used to encapsulate common logic between multiple default methods within the same interface.

4. Interface vs. Abstract Class

This is the **#1 most asked question**. You must answer this with precision.

Feature	Interface	Abstract Class
Inheritance	A class can implement many.	A class can extend only one.
Variables	Only public static final (constants).	Can have instance variables (state).
Methods	Mostly abstract (until Java 8).	Can be abstract or concrete.
Constructor	Cannot have constructors.	Can have constructors (to initialize state).
Relationship	Defines a Capability ("Can-Do").	Defines an Identity ("Is-A").

Real-world analogy: > * Abstract Class: A **Vehicle** (It has an engine, wheels, and a VIN number).

- Interface: **GPS_Enabled** (A phone can have GPS, a car can have GPS, and a watch can have GPS. They aren't the same "thing," but they share a "capability").

5. Important Interface Types

A. Marker Interface

An interface with **no methods or fields**. It tells the JVM that a class has a specific property.

- **Examples:** Serializable, Cloneable, Remote.
- **Interview Question:** "How do you create your own Marker Interface?" * Answer: Just create an empty interface and use instanceof to check for it in your logic.

B. Functional Interface (Java 8+)

An interface that contains **exactly one abstract method**. These are the foundation for **Lambda Expressions**.

- **Examples:** Runnable, Callable, ActionListener.
- **Annotation:** @FunctionalInterface (Optional, but best practice).

6. Potential Interview Scenarios

Scenario: The Diamond Problem in Interfaces

- **Question:** "If Interface A and Interface B both have a default void display() method, and Class C implements both, what happens?"
- **Answer: Compiler Error.** Java won't know which one to use.
- **Solution:** Class C **must** override the display() method and manually resolve the conflict (e.g., InterfaceA.super.display();).

Scenario: Variable Modification

- **Question:** "Can I change the value of a variable defined in an interface?"
- **Answer: No.** They are final by default. Attempting to change them will result in a compile-time error.

1. Nested Classes (Classes within Classes)

Interviewer: "Why would you ever put a class inside another class?"

The Goal: It's all about **Encapsulation**. If a class is only useful to one other class, keep it inside.

Type	Description	Key Characteristic
Static Nested Class	A static class inside another.	Can be instantiated without an outer object.
Inner Class	A non-static class inside another.	Must have an instance of the outer class to exist.
Local Inner Class	Defined inside a method .	Scope is limited to that method only.
Anonymous Inner Class	A class without a name.	Used to override methods of a class/interface on the fly.

Interview Trick: "Can a static nested class access private members of the outer class?"

Answer: Yes! Even if they are private, because it is still "inside" the outer class. However, it can only access **static** private members.

2. The final Keyword (3 Levels of Restriction)

Interviewers love to ask about final because it behaves differently depending on where it's placed.

1. **Final Variable:** Creates a constant. Its value cannot be changed once assigned.
2. **Final Method:** Prevents **Method Overriding**. You use this when you don't want a subclass to change the "core logic" of a function.
3. **Final Class:** Prevents **Inheritance**. You cannot extend a final class (e.g., the String class is final for security and performance).

3. Object Cloning (Shallow vs. Deep Copy)

If you just do `Student s2 = s1;`, you aren't copying the object; you are just creating two pointers to the same memory. To actually duplicate an object, we use **Cloning**.

A. Shallow Copy (Default)

It creates a new object, but if the original object contains references to other objects (like a List or an Address object), the copy will point to the **same** internal objects.

- *Risk:* If you change the address in the copy, it changes in the original too.

B. Deep Copy

You manually override the `clone()` method to create new instances of the internal objects as well.

- *Result:* The two objects are 100% independent.

4. Possible Interview Questions

1. **What is the "Blank Final" variable?**
 - *Answer:* A final variable that is not initialized at the time of declaration. It **must** be initialized in the constructor. This allows each instance of a class to have a different "constant" value.
2. **Why is the String class immutable and final?**
 - *Answer:* 1. **Security:** Strings are used for usernames/passwords. 2. **Caching:** It allows the String Constant Pool to work. 3. **Thread Safety:** Multiple threads can share a String without fear of change.
3. **What is the Cloneable interface?**
 - *Answer:* It is a **Marker Interface**. If you don't implement it and call `clone()`, Java throws a `CloneNotSupportedException`.

5. The "Logic" Challenge

Scenario: You have a class User with a final List of Orders.

```
final List<Order> orders = new ArrayList<>();
```

Question: Can you add a new order to this list?

Answer: Yes. The Logic: final on an object reference means you cannot point orders to a **new** List. However, the *content* of the object it points to can still be modified. This is a very common trick question!

1. Java I/O (Input and Output)

Java I/O is based on the concept of **Streams**. A stream is a continuous flow of data.

Byte Streams vs. Character Streams

This is the fundamental "Which one do I use?" question.

Feature	Byte Streams	Character Streams
Classes	InputStream, OutputStream	Reader, Writer
Data Type	8-bit bytes (Images, Videos, Binary)	16-bit Unicode (Text files)
Logic	Processes data "raw."	Processes data specifically for text/languages.

2. Serialization: The "Object Freezer"

Serialization is the process of converting an object's state into a byte stream. **Deserialization** is the reverse—recreating the object in RAM from that byte stream.

Why do we need it?

- **Persistence:** To save an object to a file.
- **Communication:** To send an object over a network (like in Microservices).

How to implement it:

A class must implement the `Serializable` interface.

Remember: It is a **Marker Interface** (it has no methods). It just tells the JVM, "I am okay with being turned into bytes."

3. The transient Keyword

This is a "Must Know" for interviews.

If you have a `User` object with a username and a password, you might want to save the username but **not** the password for security reasons.

- **Logic:** Any variable marked as transient will **not** be serialized. During deserialization, it will be initialized with its default value (e.g., null for objects, 0 for ints).

4. Potential Interview Questions

1. What is serialVersionUID?

- *Answer:* It is a unique ID for each version of a class. If you serialize an object and then change the class (add a field), the serialVersionUID helps the JVM ensure the file and the class are compatible during deserialization.

2. Can we serialize a static variable?

- *Answer: No.* Serialization is for saving the "state" of an object. Static variables belong to the class, not the object, so they are not part of the serialized stream.

3. What happens if a parent class is NOT serializable but the child is?

- *Answer:* The child can still be serialized. However, during deserialization, the parent's constructor will run to initialize the parent's fields, whereas the child's constructor will not run (as its state is being restored from the stream).

4. What is the difference between Externalizable and Serializable?

- *Answer:* `Serializable` is automatic (JVM does the work). `Externalizable` gives the developer **complete control** via `writeExternal()` and `readExternal()` methods, but it's harder to implement.

5. Modern I/O: The NIO (New I/O)

If the company is high-tech, they might ask about `java.nio`.

- **Classic I/O:** Blocking (One thread per connection). Like a waiter waiting for a customer to finish reading the menu.
- **NIO:** Non-blocking/Buffer-oriented. Like a waiter taking orders from 5 tables and only going back when a bell rings. It's much faster for high-traffic apps.

The "Final Boss" Interview Scenario:

Interviewer: "I have a class `Employee` that implements `Serializable`. Inside it, there is a field `Department dept`. If `Department` does **not** implement `Serializable`, what happens when I try to save an `Employee`?"

The Answer: It will throw a `NotSerializableException` at runtime.

The Fix: 1. Make `Department` implement `Serializable`. 2. Mark the `dept` field as transient (if you don't need to save it).

1. What is Reflection?

Reflection is a feature in Java that allows an executing Java program to examine or "introspect" upon itself, and manipulate internal properties of the program.

- **The Power:** You can look at a class at runtime and find out its methods, fields, and constructors, even if they are marked as **private**.
- **The Location:** All reflection classes are in the `java.lang.reflect` package.

2. Why use Reflection? (The Framework Secret)

If reflection is "dangerous" (it breaks encapsulation), why do we use it?

- **Spring Framework:** Uses reflection to look at your classes and see if they have `@Autowired` or `@Service` annotations.
- **Unit Testing (JUnit):** Uses reflection to find all methods marked with `@Test` and execute them.
- **IDEs (IntelliJ/Eclipse):** When you type object., the list of methods that pops up is generated via reflection.

3. How to use it? (The Mechanics)

To reflect on a class, you first need its `Class` object. There are three ways to get it:

1. `Class c = MyClass.class;`
2. `Class c = obj.getClass();`
3. `Class c = Class.forName("com.example.MyClass");`

Breaking Encapsulation (The Interviewer's Favorite)

Java

```
Field field = MyClass.class.getDeclaredField("privatePassword");
field.setAccessible(true); // The magic line that breaks the "private" barrier
field.set(obj, "newPassword");
```

4. The Downsides (The "When NOT to use" Question)

An interviewer will ask: *"If Reflection is so powerful, why don't we use it for everything?"*

1. **Performance Overhead:** Since types are resolved at runtime, the JVM cannot perform certain optimizations. It is much slower than direct code.
2. **Security Risk:** It can break the safety of private data (as shown above).
3. **Maintenance:** Code that uses reflection is harder to read and debug. If a method name changes, the compiler won't catch it—the app will just crash at runtime.

5. Potential Interview Questions

1. **Can Reflection break a Singleton?**
 - *Answer: Yes.* Even if the constructor is private, reflection can call `setAccessible(true)` and create a second instance. (To prevent this, you can throw an exception inside the constructor if an instance already exists).
2. **What is the "Class" class in Java?**
 - *Answer:* It is a class that represents the metadata of other classes. Every time a class is loaded, the JVM creates a `java.lang.Class` object to keep track of it.
3. **Difference between `getMethods()` and `getDeclaredMethods()`?**
 - *Answer:* `getMethods()` returns all **public** methods (including inherited ones). `getDeclaredMethods()` returns **all** methods (private, protected, public) but **only** from the current class.

6. Real-World Scenario: The Plugin System

Question: "Imagine you are building a system that allows users to upload their own 'Plugin' JAR files. You don't know the names of their classes or methods beforehand. How will your main app run their code?" **Answer: Reflection.** I would scan the JAR for classes implementing a Plugin interface, find their names, and use `Class.forName()` and `newInstance()` to run them dynamically.

Are we ready for the Final Verdict?

We have officially covered:

- **Basics & Architecture** (JVM, JDK, JRE)
- **OOPS Pillars** (Encapsulation, Inheritance, Polymorphism, Abstraction)
- **Memory Management** (Stack vs Heap, Garbage Collection)
- **Collections Framework** (List, Set, Map Internals)
- **Exception Handling** (Checked, Unchecked, Finally)
- **Multithreading** (Synchronization, Volatile, Lifecycle)
- **Java 8+ Features** (Streams, Lambdas, Optional)
- **Design Patterns** (Singleton, Factory, Strategy, etc.)
- **I/O & Serialization**
- **Reflection**

Access Specifiers

In Java, **Access Specifiers** (also known as access modifiers) are keywords used to define the **scope** and **visibility** of classes, constructors, variables, and methods. They are the primary tools for achieving **Encapsulation**. Here is the detailed breakdown of how they work and the scenarios you'll face in interviews.

1. The Access Level Hierarchy

There are four levels of access in Java. If you don't specify any modifier, Java applies the **Default** level.

Modifier	Class	Package	Subclass (Inheritance)	World (Everywhere)
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes	No

Modifier	Class	Package	Subclass (Inheritance)	World (Everywhere)
default (no keyword)	Yes	Yes	No	No
private	Yes	No	No	No

2. Detailed Breakdown & Real-World Scenarios

A. Private (The Most Restrictive)

- **Visibility:** Only within the **same class**.
- **Use Case:** Use this for all data members (variables).
- **Scenario:** In a Bank class, the balance should be private. You don't want other classes to change it directly; they must go through deposit() or withdraw() methods.

B. Default (Package-Private)

- **Visibility:** Only within the **same package**.
- **Use Case:** When you have a group of classes working together as a "module" and you want them to see each other's data, but hide it from the rest of the world.
- **Scenario:** A PaymentHelper class that should only be accessible to other classes inside the com.app.payment package.

C. Protected (The Inheritance Special)

- **Visibility:** Same package **AND** subclasses in different packages.
- **Use Case:** Use this when you want to allow child classes to access parent members but keep them hidden from unrelated classes.
- **Scenario:** A BaseUI class has a render() method. You want the MobileUI and WebUI subclasses (even in different packages) to use it, but not a random Database class.

D. Public (The Universal Access)

- **Visibility:** Everywhere.
- **Use Case:** Use this for methods that provide services to the user and for the main class.
- **Scenario:** The main method must be public so the JVM can call it from outside the package.

3. Potential Interview "Tricky" Questions

Q1: Can a top-level class be private or protected?

Answer: No. A top-level class can only be public or default. Only **Inner Classes** (nested classes) can be private or protected. If you make a top-level class private, no one could ever use it, making it useless.

Q2: What is the "Access Modifier Rule" in Method Overriding?

Answer: This is a high-frequency question. When you override a method, the subclass method **cannot** have a more restrictive access modifier than the parent method.

- *Example:* If a parent method is protected, the overriding method in the child can be protected or public, but **not** private or default.

Q3: Why is there no "keyword" for default access?

Answer: It is a legacy design choice in Java. It is often called "Package-Private." If you don't type anything, Java assumes you want the member to be visible only within that package.

Q4: If a constructor is private, can we create an object of that class?

Answer: Yes, but only from within that same class. This is exactly how the **Singleton Design Pattern** works—the class provides a static method that calls the private constructor.

Constructors

1. What is a Constructor? (The Essentials)

A constructor is used to **initialize the state** of an object (assigning values to variables).

Key Rules for Constructors:

1. **Name Match:** It must have the exact same name as the Class.
2. **No Return Type:** It does not have a return type—not even void.
3. **Automatic Call:** It is called the moment you use the new keyword.
4. **Modifiers:** It can have access modifiers (public, private, etc.), but it cannot be abstract, static, final, or synchronized.

2. Types of Constructors

A. Default Constructor (The "Hidden" One)

If you do not write any constructor in your class, the Java compiler **automatically inserts** a no-argument constructor for you.

- **Purpose:** To provide default values to the object (like 0 for int, null for String).
- **Note:** If you write *any* constructor yourself, the compiler will **not** provide the default one.

B. No-Argument Constructor

This is a constructor written by the programmer that takes no parameters.

```
Java
class Student {
    String name;
    Student() { // No-arg constructor
        name = "Unknown";
    }
}
```

C. Parameterized Constructor

Used to initialize the object with **specific values** at the time of creation.

```
Java
class Student {
    String name;
    Student(String n) { // Parameterized
        name = n;
    }
}
```

// Usage: Student s1 = new Student("Gemini");

D. Copy Constructor (The "Cloner")

Java doesn't have a built-in copy constructor like C++, but we can create one to copy the values of one object into another.

```
Java
class Student {
    String name;
    Student(Student other) { // Copy Constructor
        this.name = other.name;
    }
}
```

3. Constructor Overloading

Just like method overloading, you can have multiple constructors in the same class with different parameter lists.

```
Java
class Car {
    String brand;
    int year;

    Car() { brand = "Generic"; year = 2020; }
    Car(String b) { brand = b; year = 2024; }
    Car(String b, int y) { brand = b; year = y; }
}
```

4. Constructor Chaining (this() and super())

This is a frequent "level-up" interview topic. **Constructor Chaining** is the process of calling one constructor from another.

- **this():** Calls another constructor in the **same class**.
- **super():** Calls the constructor of the **parent class**.

Crucial Rule: this() or super() **must** be the first statement in a constructor. You cannot use both in the same constructor.

5. Potential Interview Questions

Q1: What is the difference between a Constructor and a Method?

- **Answer:** A constructor initializes an object and has no return type; a method performs an operation and must have a return type. A constructor is called implicitly; a method is called explicitly.

Q2: Can a constructor be private?

- **Answer: Yes.** We use private constructors in the **Singleton Design Pattern** or for utility classes (like java.lang.Math) where we want to prevent people from creating objects.

Q3: Is the constructor inherited?

- **Answer: No.** A subclass does not inherit its parent's constructors, but it **calls** them using super().

Q4: Can we use this inside a static method?

- **Answer:** No, because this refers to the "current object instance," and static methods belong to the class, not an instance.

6. The Execution Order (Scenario Test)

Interviewer Question: "What happens when I say new Child() if Child extends Parent?"

1. The Child constructor is called.

2. The first line of Child is implicitly super().
3. The Parent constructor executes.
4. The rest of the Child constructor executes.

DESTRUCTOR

Unlike C++, where you must manually write a destructor (~ClassName) to deallocate memory, Java uses **Automatic Garbage Collection**. However, an interviewer asking this is actually testing your knowledge of **Resource Management** and the **Object Lifecycle**.

1. The Core Concept: Java vs. C++

In C++, a destructor is called the moment an object goes out of scope. In Java, we don't know exactly *when* an object will be destroyed because the **Garbage Collector (GC)** runs on its own schedule.

Feature	C++ Destructor	Java Approach
Name	~ClassName()	No specific name / finalize() (Deprecated)
Invocation	Explicit/Automatic when out of scope.	Managed by Garbage Collector.
Memory	Manual deallocation.	Automatic deallocation.
Resources	Closed manually in destructor.	Closed via try-with-resources or finally.

2. If no Destructor, how do we clean up?

Since we can't rely on a destructor, Java provides three main ways to handle the "teardown" of an object.

A. The finally Block (The Traditional Way)

Used to ensure that a resource (like a database connection) is closed regardless of whether an exception occurred.

Java

```

FileWriter out = null;
try {
    out = new FileWriter("file.txt");
} finally {
    if (out != null) out.close(); // Manual cleanup
}

```

B. Try-with-Resources (The Modern Way)

Introduced in Java 7, any class that implements AutoCloseable can be used here. Java automatically "destructs" the resource at the end of the block.

Java

```

try (FileWriter out = new FileWriter("file.txt")) {
    // Work with file
} // 'out' is automatically closed here!

```

C. The finalize() Method (The Deprecated Way)

The Object class has a protected void finalize() method. The GC calls this before it removes an object from memory.

- **Why it's bad:** It's unpredictable, slow, and can actually prevent objects from being collected.
- **Current Status: Deprecated since Java 9.** You should tell the interviewer: *"We do not use finalize() in modern Java development."*

3. The Object Lifecycle "Death" Phase

In the "all aspects" view, you need to explain what happens when an object is ready to be destroyed:

1. **Unreachable State:** The object no longer has any references pointing to it in the **Stack**.
2. **Eligibility:** The GC marks the object as eligible for collection.
3. **Finalization:** If the object has a finalize() method, it's added to a finalization queue (this is the step that causes delays).
4. **Deallocation:** The GC reclaims the memory in the **Heap**.

4. Interview "Trap" Questions on Destructors

Q1: "Can we manually call the Garbage Collector to act like a destructor?"

- **Answer:** You can call System.gc(), but it's just a **request**, not a command. The JVM might ignore it. You cannot force the "destruction" of an object in Java.

Q2: "What is a Memory Leak in Java if there are no destructors?"

- **Answer:** A leak occurs when an object is no longer needed but is still **reachable** (e.g., a static List holding onto millions of objects). Because the reference exists, the GC won't touch it.

Q3: "If Java has no destructors, how do we handle Native (C/C++) resources?"

- **Answer:** We use the Cleaner or PhantomReference API (introduced in Java 9) to perform actions after an object becomes unreachable, which is safer and faster than finalize().

Virtual Function

In Java, every non-static, non-final, and non-private method is a Virtual Function by default. You don't need a keyword. Java's entire method-calling mechanism is built around this concept, known as Runtime Polymorphism or Dynamic Method Dispatch.

What is a Virtual Function?

A virtual function is a method whose behavior can be overridden in a subclass. The most important characteristic is that the call to the method is resolved at **Runtime**, not at Compile-time.

- **The Goal:** To allow a parent class reference to point to a child class object and execute the child's version of a method.
- **The Mechanism:** The JVM uses something called a **VTable (Virtual Table)** to look up the actual method implementation at runtime based on the actual object type, not the reference type.

Imagine you have a **Universal Remote Control** (The Parent Reference).

- **The Reference:** The remote has a button labeled PowerOn().
- **The Objects:** You can point this remote at a **Television**, a **Music System**, or an **Air Conditioner** (The Subclasses).
- **The Virtual Action:** When you press PowerOn(), the actual "action" depends on what device the remote is currently pointing at.
 - If pointing at the **TV** $\rightarrow$ Screen glows.
 - If pointing at the **AC** $\rightarrow$ Compressor starts.

In code, the remote doesn't need to know *how* the device turns on; it just calls the "virtual" powerOn() method, and the specific object handles the rest.

Interviewers often give you a code snippet like this and ask for the output:

```
class Payment {
    void process() { System.out.println("Processing generic payment"); }
}

class UPI extends Payment {
    @Override
    void process() { System.out.println("Processing UPI payment via Scanner"); }
}

class CreditCard extends Payment {
    @Override
    void process() { System.out.println("Processing Card payment via OTP"); }
}

public class Main {
    public static void main(String[] args) {
        Payment myPay; // Parent Reference

        myPay = new UPI();
        myPay.process(); // Output: Processing UPI payment via Scanner

        myPay = new CreditCard();
        myPay.process(); // Output: Processing Card payment via OTP
    }
}
```

Why is this "Virtual"?

Even though myPay is declared as a Payment type, the JVM sees that at runtime, myPay is actually holding a UPI object. Because the process() method is **virtual**, Java ignores the Payment version and runs the UPI version.

When is a Function NOT Virtual?

To crack the interview, you must know the exceptions. A method is **not** virtual (meaning it cannot be overridden/dynamically dispatched) if it is:

1. **private methods:** Not visible to subclasses.
2. **final methods:** Cannot be overridden by subclasses.
3. **static methods:** These belong to the class, not the object. This is called **Method Hiding**, not overriding.
4. **Constructors:** They are not inherited or overridden.

Q1: Does Java have a virtual keyword?

- **Answer:** No. Unlike C++, Java makes all instance methods virtual by default.

Q2: What is a "Pure Virtual Function" in Java?

- **Answer:** In Java, an **Abstract Method** (a method with no body in an abstract class or interface) is the equivalent of a C++ Pure Virtual Function. It forces the subclass to provide an implementation.

Q3: What is the benefit of Virtual Functions?

- **Answer: Decoupling.** It allows you to write code that works with a Parent class (like Shape) without needing to know which specific child (like Circle or Square) will be used later. This makes the system easily extendable.

Dynamic Binding (late binding)

In the world of Java, Dynamic Binding (also known as Late Binding) is the mechanism where a method call is resolved at runtime rather than at compile-time.

It is the technical backbone of Runtime Polymorphism.

When you call a method on an object reference, the compiler checks if that method exists in the reference type. However, it doesn't decide *which* version of the method to execute. At runtime, the JVM looks at the actual object the reference is pointing to and calls the method defined in that specific object's class.

The Prerequisites

For dynamic binding to occur, three conditions must be met:

1. **Inheritance:** There must be a Parent-Child relationship.
2. **Method Overriding:** The child class must override a method of the parent class.
3. **Upcasting:** A parent class reference must be used to point to a child class object.

Real-World Example: The "Smartphone" Scenario

Imagine you have a variable of type Smartphone.

- **Scenario:** You tell the phone to takePhoto().
- **Compile-time:** The compiler only checks: "*Does a Smartphone have a takePhoto() method?*" Yes.
- **Runtime (Late Binding):** * If the actual object is an **iPhone**, it uses the iOS camera logic.
 - If the actual object is a **Samsung**, it uses the Android camera logic.
 - If the actual object is a **Google Pixel**, it uses the HDR+ logic.

The "binding" to the specific camera logic happens **late** (when the app is actually running), not when the code was compiled.

The "VTable" Logic (How it works internally)

How does the JVM know where the child's method is? Every class has a **Virtual Method Table (VTable)**. This table contains the memory addresses of all the virtual methods for that class.

1. When a method is called, the JVM looks at the object's header to find its VTable.
2. It then looks for the method's signature in that table.
3. It jumps to that memory address and executes the code.

Why do we need it?

- **Flexibility:** You can write a single method that takes a Shape parameter and it will automatically work for Circle, Square, or Triangle without changing the code.
- **Extensibility:** You can add new child classes later, and the existing code will work perfectly with them via dynamic binding.

Static Binding

Static binding occurs when the information required to call a method or access a variable is available at **compile-time**. The compiler looks at the **Reference Type** (the class name you wrote in your code) to decide what to bind, rather than looking at the actual object created in memory.

The "Big Three" of Static Binding

In Java, there are three specific scenarios where binding is always static:

1. **static methods:** They belong to the class, not the instance.
2. **private methods:** They cannot be overridden, so the compiler knows exactly which one to call.
3. **final methods:** Since they are "final," no subclass can change them, making the call predictable.

4. **Method Overloading:** The compiler distinguishes overloaded methods by their parameter list during compilation.

2. Real-World Example: The "Static Label"

Think of a **Library**.

- **Static Binding:** Every book has a **Call Number** (like "621.382"). This number is printed on the spine at the "factory." No matter who borrows the book or where it's shelved, that number never changes. The librarian knows exactly which book it is just by looking at the label.
- **Dynamic Binding:** This is like a **"Staff Pick"** shelf. The shelf itself is generic, but the actual book on it changes every week. You only know what you're getting once you walk up to the shelf and look at the physical book (Runtime).

3. Variable Binding (The Interview "Gotcha")

One of the most important rules in Java is that **Variables are always statically bound**. This is the #1 trick question in OOPS interviews.

The Scenario:

Java

```
class Parent {  
    int speed = 50;  
}
```

```
class Child extends Parent {  
    int speed = 100;  
}
```

```
public class Test {  
    public static void main(String[] args) {  
        Parent obj = new Child();  
        System.out.println(obj.speed); // Output: 50  
    }  
}
```

Why 50? Even though the object is a Child, the reference obj is of type Parent. Since variables use **Static Binding**, the compiler looks at the reference type (Parent) and binds the call to Parent.speed.

4. Static vs. Dynamic Binding: Key Differences

Feature	Static Binding	Dynamic Binding
Time	Compile-time	Runtime
Logic Based On	Reference Type	Actual Object Type
Performance	Faster (Direct call)	Slower (VTable lookup overhead)
Keywords	static, private, final, overloading	overriding, virtual methods
Flexibility	Rigid / Fixed	Highly Flexible

5. Why is it used?

1. **Performance:** Because the binding happens at compile-time, the JVM doesn't have to waste time searching for the method in a VTable during execution. This makes static and final methods slightly faster.
2. **Security:** By using final or private, you ensure that the core logic of your method cannot be changed or "hijacked" by a subclass.
3. **Hiding:** Static methods can be **hidden** (not overridden), allowing a class to maintain its own version of a utility method without interference from inheritance.

6. Common Interview Questions

Q1: Can a static method be overridden?

- **Answer:** No. It can be **hidden**. If both Parent and Child have the same static method, the version that gets called depends on the **Reference Type** (Static Binding).

Q2: Why is method overloading an example of Static Binding?

- **Answer:** Because the compiler checks the method signatures (parameters) at compile-time to decide which version to call. The decision is made before the .class file is even generated.

Q3: Which is better for performance?

- **Answer:** Static Binding. It avoids the overhead of dynamic lookup. However, Dynamic Binding is usually preferred for the architectural flexibility it provides.

Does Java support **Multiple Inheritance**?

- **With Classes: NO.** A class cannot extend more than one class. (e.g., class C extends A, B is a compile-time error).
- **With Interfaces: YES.** A class can implement multiple interfaces, and an interface can extend multiple other interfaces.

2. WHY doesn't Java support it with Classes? (The Diamond Problem)

The primary reason is to avoid **ambiguity** and keep the language simple. This is best explained by the **Diamond Problem**.

The Scenario:

Imagine two classes, Class B and Class C, both inherit from Class A. Now, imagine Class D inherits from both B and C.

1. Class A has a method display().
2. Class B overrides display().
3. Class C also overrides display().
4. If Class D inherits from both B and C and calls display(), which version should it use? B's or C's?

This ambiguity creates "deadly diamonds." To prevent this confusion and the complexity it adds to the JVM, Java designers decided to disallow multiple inheritance with classes.

3. How does Java handle it with Interfaces?

Java allows a class to implement multiple interfaces because interfaces (traditionally) only contained **abstract methods** (method signatures without bodies).

- **The Logic:** If Interface B and Interface C both have a method print(), Class D is forced to provide its own implementation of print(). Since there is only one implementation (the one in Class D), there is no ambiguity.

The Modern Twist (Java 8+ Default Methods)

With the introduction of default methods in Java 8, interfaces can now have method bodies. This brings back the Diamond Problem!

- **The Solution:** If a class implements two interfaces that have the same default method, the compiler throws an error. The developer **must** resolve the conflict by overriding the method in the class and explicitly calling the desired interface's method using the super keyword.

Java

```
class D implements InterfaceB, InterfaceC {
    @Override
    public void display() {
        InterfaceB.super.display(); // Explicitly choosing B's version
    }
}
```

4. Multiple Inheritance in Interfaces (Interface to Interface)

While a class can only extend one class, an **interface can extend multiple interfaces**.

Java

```
interface Printable { void print(); }
interface Showable { void show(); }
```

```
interface Readable extends Printable, Showable {
    // This is perfectly valid in Java
}
```

5. Potential Interview Questions

Q1: Why does C++ support multiple inheritance but Java doesn't?

- **Answer:** Java was designed to be a simpler, safer language. Multiple inheritance leads to complex issues like the Diamond Problem and makes the memory management (vtable) more complicated. Java designers felt that the benefits didn't outweigh the risks.

Q2: How can we achieve the benefits of Multiple Inheritance in Java without using Interfaces?

- **Answer:** Use **Composition**. Instead of inheriting from two classes, make the class contain instances of those two classes. This is known as the "Has-A" relationship.

Q3: Can a class extend one class and implement multiple interfaces at the same time?

- **Answer:** Yes. The syntax is class A extends B implements C, D. Note that the extends keyword must come before implements.

The **Diamond Problem** is a famous conflict that occurs in languages that support multiple inheritance (like C++). It is the primary reason why **Java does not allow a class to extend more than one class**.

The name comes from the shape of the inheritance diagram that is created when this conflict arises.

1. The Scenario

Imagine an inheritance hierarchy consisting of four classes:

1. **Class A (Grandparent):** Has a method called `execute()`.
2. **Class B (Parent 1):** Extends Class A and **overrides** the `execute()` method with its own logic.
3. **Class C (Parent 2):** Also extends Class A and **overrides** the `execute()` method with its own different logic.
4. **Class D (Child):** Attempts to extend **both** Class B and Class C.

2. The Conflict (The Ambiguity)

If you try to call `objD.execute()`, the JVM faces a massive dilemma:

- Should it run the version of `execute()` from **Class B**?
- Should it run the version of `execute()` from **Class C**?

Because Class D has two parents that provide two different versions of the same method, there is no way for the compiler to decide which one is "correct." This ambiguity is the **Diamond Problem**.

3. How Java Avoids It

Java designers decided that the complexity of resolving these conflicts wasn't worth the risk.

- **Classes:** Java strictly allows a class to have only **one superclass**. By preventing a class from having two parents, the diamond shape can never be formed, and the ambiguity is killed at the root.
- **Interfaces:** Originally, interfaces only had abstract methods (no bodies). Even if a class implemented two interfaces with the same method signature, the class was forced to provide its *own* single implementation. No body, no conflict.

4. The "Modern" Diamond Problem (Java 8+)

When Java 8 introduced **Default Methods** (methods with bodies inside interfaces), the Diamond Problem returned!

The Scenario:

If Interface B and Interface C both have a default void `start()` method, and Class D implements both:

Java

```
interface B {  
    default void start() { System.out.println("B starts"); }  
}
```

```
interface C {  
    default void start() { System.out.println("C starts"); }  
}
```

```
class D implements B, C {  
    // COMPILER ERROR! Which start() should I use?  
}
```

The Solution:

Java forces the developer to **manually resolve the conflict**. If you don't override the method in the child class, the code won't compile. You must explicitly choose which parent to follow:

Java

```
class D implements B, C {  
    @Override  
    public void start() {  
        B.super.start(); // Explicitly tells Java to use B's version  
    }  
}
```

Inheritance and Composition

both **Inheritance** and **Composition** are used to establish relationships between classes. However, they serve very different purposes.

An interviewer will often ask: *"Why is it said that you should favor Composition over Inheritance?"* To answer this, you must understand the "Is-A" vs. "Has-A" logic.

1. Inheritance ("Is-A" Relationship)

Inheritance is when one class derives from another to reuse its code and behavior. It creates a strict hierarchy.

- **Mechanism:** Uses the `extends` keyword.
- **Concept:** A subclass is a specialized version of the superclass.
- **Example:** A Car **is a** Vehicle. A SavingAccount **is a** BankAccount.

2. Composition ("Has-A" Relationship)

Composition is when one class contains an instance of another class as a member variable. Instead of *being* the other class, it *uses* the other class.

- **Mechanism:** Declaring an instance of another class as a field.
- **Concept:** Building complex objects by combining simpler ones.
- **Example:** A Car **has an** Engine. A Library **has a** List<Book>.

0

3. Key Differences: The Comparison

Feature	Inheritance	Composition
Relationship	Is-A (Strict)	Has-A (Flexible)
Binding	Static (Compile-time)	Dynamic (Runtime)
Coupling	Tight Coupling: Changes in parent affect all children.	Loose Coupling: Components are independent.
Visibility	Subclass can see protected members.	Container only sees public interface.
Flexibility	Cannot change the parent at runtime.	Can swap internal objects at runtime.

4. Why "Favor Composition over Inheritance"?

This is the "Golden Rule" of design. Here is the scenario-based reason why:

The Problem with Inheritance (Tight Coupling)

Imagine you have a class Bird with a fly() method. You create a subclass Eagle. Everything is fine. Now you create a subclass Ostrich.

- **The Issue:** An Ostrich is a Bird, but it **cannot fly**. Because you used inheritance, Ostrich is now forced to have a fly() method it doesn't need. This is known as the **LSP (Liskov Substitution Principle) violation**.

The Solution with Composition (Flexibility)

Instead of forcing every bird to inherit fly(), you create a FlyingBehavior class.

- **The Logic:** You give the Eagle an instance of FastFlyingBehavior. You give the Ostrich an instance of NoFlyingBehavior (or simply don't give it one).
- **Result:** You can change the behavior of an object at runtime without breaking the class hierarchy.

5. Aggregation vs. Composition

In interviews, you might be asked to go deeper into "Has-A" relationships. Composition is actually a stricter form of Aggregation.

1. **Aggregation (Weak Has-A):** The child can exist independently of the parent.
 - *Example:* A Department has a Teacher. If the Department is deleted, the Teacher still exists.
2. **Composition (Strong Has-A):** The child cannot exist without the parent.
 - *Example:* A House has a Room. If the House is destroyed, the Rooms are destroyed too.

Coupling and Cohesion

Coupling and **Cohesion** are the two primary metrics used to measure the quality of a design. If you want to build a system that is easy to maintain, you aim for **Low Coupling** and **High Cohesion**.

1. Cohesion (Internal Strength)

Cohesion refers to how **focused** a single module or class is. It measures how closely the responsibilities within a class are related to each other.

- **High Cohesion (Good):** A class does exactly one thing and does it well. All its methods relate to its primary purpose.
- **Low Cohesion (Bad):** A class is a "Jack of all trades." It handles database logic, UI rendering, and email sending all at once. This is often called a **"God Object."**

Real-World Example:

Imagine a **Kitchen**. A highly cohesive kitchen has tools only for cooking (knives, stove, pans). If you start keeping your car's spare tires and a sewing machine in the kitchen, it becomes **low cohesion**—it's harder to find what you need because the items aren't related.

2. Coupling (External Dependency)

Coupling refers to how **interdependent** two or more modules are. It measures how much one class "knows" about another.

- **Low/Loose Coupling (Good):** Classes are independent. You can change the code in Class A without breaking Class B. They communicate through simple interfaces.
- **High/Tight Coupling (Bad):** Classes are highly dependent. If you change a variable name in Class A, you have to manually update code in Class B, C, and D.

Real-World Example:

- **Tight Coupling:** An old-fashioned desk lamp where the bulb is soldered into the socket. To change the bulb, you have to take the whole lamp apart.
- **Loose Coupling:** A standard light socket. You can swap a 40W bulb for a 100W bulb or an LED bulb effortlessly because they follow a standard interface (the screw thread).

3. Comparison Table

Metric	Goal	Focus	Impact of Change
Cohesion	High	Inside the module (Internal)	High cohesion makes code easier to understand and reuse.
Coupling	Low	Between modules (External)	Low coupling makes code easier to modify without "side effects."

4. Why do they matter in Interviews?

An interviewer will ask: *"How do you achieve Low Coupling and High Cohesion in Java?"*

To achieve High Cohesion:

- Follow the **Single Responsibility Principle (SRP)**: Each class should have only one reason to change.
- If a class is getting too big, split it. For example, move the "Validation" logic out of the "User" class and into a "UserValidator" class.

To achieve Low/Loose Coupling:

- **Encapsulation:** Use private variables so other classes can't mess with internal state.
- **Program to Interfaces:** Instead of `ArrayList list = new ArrayList()`, use `List list = new ArrayList()`. This allows you to swap `ArrayList` for `LinkedList` later without changing the rest of your code.
- **Dependency Injection:** Don't let a class create its own dependencies; "inject" them from the outside.

5. The Scenario Test

Interviewer: *"If I have a class that calculates taxes, saves the result to a database, and prints a receipt, is this a good design?"*

Your Answer: "No. This class has **Low Cohesion** because it is handling three unrelated tasks (Calculation, Persistence, and Formatting). It also likely has **High Coupling** because it depends on the specific Database class and the specific Printer class. I would refactor this into three separate, highly cohesive classes."

SOLID principles

If Low Coupling and High Cohesion are the goals, the SOLID Principles are the manual on how to get there. These five design principles, introduced by Robert C. Martin ("Uncle Bob"), are the gold standard for writing maintainable and scalable object-oriented software.

In a placement interview, being able to explain these with examples is what separates a beginner from a professional.

1. S - Single Responsibility Principle (SRP)

"A class should have one, and only one, reason to change."

The Concept: A class should do one thing and do it well. If a class has multiple responsibilities, they become coupled. A change to one responsibility might break the other.

- Bad Example: A User class that stores user data and saves it to a database and sends email notifications.
- Good Example: Three separate classes: User (data), UserRepository (database), and EmailService (notifications).

2. O - Open/Closed Principle (OCP)

"Software entities should be open for extension, but closed for modification."

The Concept: You should be able to add new functionality without changing existing code. This prevents bugs in code that is already tested and working.

- Bad Example: Using a massive if-else or switch block to handle different payment types (Credit Card, PayPal). Every time you add a new type, you must modify the existing class.
- Good Example: Use an interface `PaymentMethod`. To add a new payment type, just create a new class that implements it. No existing code is touched.

3. L - Liskov Substitution Principle (LSP)

"Objects of a superclass should be replaceable with objects of its subclasses without breaking the application."

The Concept: A subclass should truly be a specialized version of its parent. It should not override parent behavior in a way that changes the expected logic.

- The Classic "Ostrich" Problem: If you have a class `Bird` with a method `fly()`, and you create a subclass `Ostrich`, you have a problem. An Ostrich is a bird, but it can't fly. If your code expects any "Bird" to be able to fly, it will crash when it meets an Ostrich.
- The Fix: Don't put `fly()` in the `Bird` class if not all birds fly. Use an interface `Flyable` instead.

4. I - Interface Segregation Principle (ISP)

"No client should be forced to depend on methods it does not use."

The Concept: It is better to have many small, specific interfaces than one large, "fat" interface.

- **Bad Example:** A SmartDevice interface with methods print(), fax(), and scan(). If you have a simple Printer class, it is forced to implement fax() and scan() with empty bodies.
- **Good Example:** Split them into Printer, Faxer, and Scanner interfaces. A class only implements what it actually needs.

5. D - Dependency Inversion Principle (DIP)

"Depend on abstractions, not on concretions."

The Concept: High-level modules should not depend on low-level modules. Both should depend on interfaces. This is the heart of Loose Coupling.

- **Bad Example:** A Car class that creates a new PetrolEngine() inside its constructor. The Car is now tightly coupled to a Petrol Engine. You can't easily change it to an Electric Engine.
- **Good Example:** The Car depends on an Engine interface. You "inject" the specific engine type (Petrol or Electric) into the Car from the outside.

SOLID Summary Table

Principle	Key Takeaway	Goal
SRP	One class = One job.	High Cohesion.
OCP	Add new code, don't change old code.	Stability.
LSP	Child must behave like Parent.	Reliability.
ISP	Don't force unnecessary methods.	Leaner code.
DIP	Use Interfaces, not concrete classes.	Low Coupling.

Scenario-Based Interview Question

Interviewer: "If I have a Document class and I add a method printToPDF() and later I add printToHTML(), which principle am I violating?"

Your Answer: "You are violating the Open/Closed Principle (OCP) because you are modifying the existing class every time you want a new format. You might also be violating SRP if the Document class is now handling 'Formatting' logic. Instead, I would create a DocumentPrinter interface and implement PDFPrinter and HTMLPrinter separately."

How are you feeling about these principles? Ready for a mock interview on these, or should we tackle "Collections Framework" next?

Deep Copy vs Shallow Copy

In Java, copying an object isn't as simple as using the = operator. When you assign one object to another (objB = objA), you aren't copying the data; you are just creating two "pointers" to the same memory location.

To actually create a duplicate, we use **Cloning**. This is where the distinction between **Shallow** and **Deep** copy becomes critical.

1. Shallow Copy (The Default)

A shallow copy creates a new instance of the class and copies all the field values. However, if those fields are **references to other objects** (like a List, an Array, or a custom Address class), only the **memory address** is copied.

- **The Result:** Both the original and the copy point to the same internal objects.
- **The Risk:** If you modify the internal object via the copy, the change reflects in the original too.
- **Default Behavior:** The Object.clone() method in Java performs a shallow copy by default.

2. Deep Copy (The Complete Duplicate)

A deep copy creates a new instance of the class and **recursively** copies all internal objects. It does not share any references with the original.

- **The Result:** The original and the copy are 100% independent.
- **The Implementation:** You must manually override the clone() method (or use a Copy Constructor) to create new instances of every nested object.
- **The Cost:** It is more expensive in terms of CPU and memory because you are creating many new objects.

3. Comparison Table

Feature	Shallow Copy	Deep Copy
Independence	Shared dependency on internal objects.	Completely independent.
Speed	Very fast.	Slower (due to multiple allocations).

Feature	Shallow Copy	Deep Copy
Implementation	Easy (super.clone()).	Complex (manual instantiation of members).
Memory	Lower memory usage.	Higher memory usage.
Use Case	Objects with only primitive fields.	Complex objects with nested collections/classes.

4. Real-World Scenario: The "User Profile"

Imagine a User class that contains an Address object.

- **Shallow Copy Scenario:** You copy "User A" to "User B." Then, User B moves to a new house and updates their address. Because it's a shallow copy, **User A's address changes too!** This is usually a bug.
- **Deep Copy Scenario:** You copy "User A" to "User B." When User B updates their address, it only changes for them. User A remains at the old address. This is the desired behavior for independent entities.

5. How to Implement Deep Copy in Java?

A. Manual Method (Overriding clone)

Java

```
class Address { String city; }
```

```
class User implements Cloneable {
    Address addr = new Address();
```

```
    public Object clone() throws CloneNotSupportedException {
        User copy = (User) super.clone(); // Shallow copy first
        copy.addr = new Address();      // Manually create new nested object
        copy.addr.city = this.addr.city;
        return copy;
    }
}
```

B. Serialization Trick

A "lazy" but effective way to do a deep copy is to **Serialize** the object into a byte stream and then **Deserialize** it back. This automatically creates a completely new object tree, though it is slower than manual cloning.

Interview Tip: "The Cloneable Marker"

If an interviewer asks, *"Why do we need to implement Cloneable?"*

Answer: Cloneable is a **Marker Interface**. It doesn't have any methods, but it tells the JVM that the class is "allowed" to be cloned. If you call clone() on a class that doesn't implement it, Java throws a CloneNotSupportedException.

FINAL VS FINALLY VS FINALIZE

Final is a keyword/modifier, **Finally** is a block in exception handling, and **Finalize** is a method for cleanup.

1. final (The Modifier)

final is a keyword used to apply **restrictions**. It can be applied to variables, methods, and classes.

- **Final Variable:** Once assigned, the value cannot be changed (it becomes a **constant**).
- **Final Method:** Prevents **Method Overriding**. A subclass cannot change the logic of this method.
- **Final Class:** Prevents **Inheritance**. You cannot extend a final class (e.g., the String class is final).

2. finally (The Cleanup Block)

finally is a block used in **Exception Handling**. It is placed after the try and catch blocks.

- **Purpose:** It is used to execute "cleanup" code (like closing a database connection or a file) regardless of whether an exception occurred or was handled.
- **Execution Rule:** It **always** executes, even if there is a return statement in the try block.
- **Exception:** It won't execute only if the JVM is force-closed via System.exit(0) or crashes.

3. finalize() (The Cleanup Method)

finalize() is a protected method defined in the Object class.

- **Purpose:** Historically, it was used to perform cleanup operations on an object just before it was destroyed by the **Garbage Collector**.
- **Status: Deprecated since Java 9.** It is considered unreliable because you never know exactly *when* the Garbage Collector will run.

- **Modern Replacement:** For cleaning up resources, we now use try-with-resources or the Cleaner API.

Comparison Summary

Feature	final	finally	finalize()
Type	Keyword / Modifier	Block (Part of try-catch)	Method (of Object class)
Used with	Variables, Methods, Classes	Exception Handling	Objects (before GC)
Main Goal	Restriction / Immutability	Guaranteed Cleanup	Object Disposal Logic
When it runs	During code execution	After try/catch completes	Just before Garbage Collection

Interview "Tricky" Questions

Q1: Can we execute a try block without a catch block?

- **Answer:** Yes, as long as there is a finally block following it. This is useful when you don't want to handle the exception yourself but must ensure resources are closed.

Q2: What is a "Blank Final" variable?

- **Answer:** A final variable that is not initialized at the time of declaration. It **must** be initialized in the constructor of the class.

Q3: Does finally run if the try block has a return 10;?

- **Answer: Yes.** The JVM will execute the finally block before actually returning the value to the caller. If finally also has a return statement, it will override the one in try.

Q4: Is finalize() a destructor?

- **Answer:** No. Java has no destructors because of automatic garbage collection. finalize() was a poor attempt to mimic them and is now deprecated.

The static Keyword

In Java, static means "belonging to the class" rather than "belonging to an instance (object)." When you mark something as static, it is initialized only **once** when the class is loaded.

A. Static Variables

- **Shared Memory:** All objects of that class share the same variable.
- **Storage:** They are stored in the **Method Area** (part of the Heap in modern JVMs), not with individual objects.
- **Use Case:** To represent common properties (e.g., collegeName for a Student class).

B. Static Methods

- **No Object Needed:** Can be called using the class name (Math.sqrt()).
- **Restriction:** They **cannot** access non-static (instance) variables or methods because static methods exist before any object is even created.

C. Static Block

- A block of code that runs **only once** when the class is first loaded into memory. Used for complex static initializations.

2. String vs. StringBuilder vs. StringBuffer

This is a high-frequency question regarding **Immutability** and **Thread Safety**.

A. String (The Immutable King)

- **Immutability:** Once created, its value cannot be changed. Any "modification" actually creates a **new** String object.
- **Storage:** Uses the **String Constant Pool (SCP)** to save memory.
- **Use Case:** When data won't change often (usernames, keys).

B. StringBuffer (The Safe Worker)

- **Mutable:** You can change the sequence of characters without creating new objects.
- **Thread-Safe:** All its methods are synchronized. This means multiple threads can use it without data corruption.
- **Performance:** Slower than StringBuilder due to the overhead of synchronization.

C. StringBuilder (The Fast Worker)

- **Mutable:** Just like StringBuffer.
- **NOT Thread-Safe:** Its methods are not synchronized.
- **Performance:** The **fastest** of the three. It is the preferred choice for single-threaded operations (like building a long string in a loop).

3. Comparison Table

Feature	String	StringBuffer	StringBuilder
Storage Area	String Constant Pool	Heap Memory	Heap Memory

Feature	String	StringBuffer	StringBuilder
Modifiability	Immutable	Mutable	Mutable
Thread Safe	Yes (because it's immutable)	Yes	No
Performance	Slowest (if modified often)	Medium	Fastest
Introduction	JDK 1.0	JDK 1.0	JDK 1.5

4. Potential Interview Questions

Q1: Why is main method static?

- **Answer:** So that the JVM can call it without creating an instance of the class. If it weren't static, there would be a "chicken and egg" problem: you'd need an object to run the method that creates the objects.

Q2: What is the String Constant Pool (SCP)?

- **Answer:** It is a special memory area in the Heap. When you create a string literal (String s = "Java"), the JVM checks the pool. If it exists, it returns a reference; if not, it creates a new one. This saves massive amounts of memory.

Q3: When should I use StringBuilder over String?

- **Answer:** Use StringBuilder when you are performing many modifications (like appending in a loop). Using String in a loop would create thousands of temporary objects, triggering the Garbage Collector and slowing down your app.

Q4: Can we override a static method?

- **Answer: No.** This is called **Method Hiding**. Since static methods are bound at compile-time (Static Binding), the version called depends on the reference type, not the actual object.

The immutability of Strings is one of the most intentional design choices in Java. It wasn't just a random decision; it was built this way to solve three major problems: **Security, Memory, and Performance**.

Here is the "why" broken down into the categories an interviewer expects to hear:

1. The String Constant Pool (Memory Efficiency)

Java uses a special memory area called the **String Constant Pool (SCP)**.

- **The Logic:** If ten different variables all have the value "Java", they all point to the same single object in the pool.
- **The Immutability Link:** If Strings were mutable, and you changed one variable to "C++", all other nine variables would suddenly change to "C++" as well. This would create massive data corruption. Immutability makes the pool safe to use.

2. Security (The "Most Important" Reason)

Strings are used everywhere in critical system operations.

- **Network & Database:** When you connect to a database, you pass the URL, username, and password as Strings.
- **File Systems:** File paths are represented as Strings.
- **The Risk:** If Strings were mutable, an attacker could pass a "safe" file path, pass the security check, and then **modify the string content** to point to a sensitive file (like /etc/passwd) before the system actually opens it. Because Strings are immutable, once the security check passes, the value is guaranteed not to change.

3. Thread Safety

Because Strings cannot change, they are **inherently thread-safe**.

- Multiple threads can share a single String instance without the need for synchronized blocks or locks.
- There is no risk of one thread changing the value while another thread is reading it. This significantly boosts performance in multi-threaded applications.

4. Caching HashCodes

In Java, Strings are frequently used as keys in **HashMaps** and **HashSets**.

- When a String is created, its hashCode is calculated and cached.
- Because the string is immutable, the hashCode will never change. This means the HashMap doesn't have to re-calculate it every time you look up a value, making String-based lookups extremely fast.

5. Class Loading

The Java ClassLoader uses Strings to find and load classes. If a String were mutable, an attacker could change the name of the class being loaded from java.io.Reader to their own malicious com.hacker.MaliciousReader, compromising the entire JVM.

Summary for your Placement Notes:

Reason	Benefit
SCP Caching	Saves massive amounts of Heap memory.
Security	Prevents "Parameter Tampering" in DB/Network/File connections.
Thread Safety	Safe sharing between threads without synchronization.
Hash Consistency	Permanent hashcode for fast HashMap performance.

The "How" (Pro-tip for Interviews)

If they ask **how** Java makes it immutable:

1. The String class is marked as **final** (so it cannot be subclassed/overridden).
2. The internal character array (char[] value) is marked as **private** and **final**.
3. The class does not provide any **setter** methods to modify the data.

IMPORTANT MAP TOPICS AND QUESTIONS

To understand Hashtable, you have to look at it as the "older, stricter sibling" of HashMap. While it is rarely used in modern development, it is a frequent interview topic because it tests your knowledge of **Thread Safety** and **Legacy Java**.

1. Internal Working: The "Buckets and Chains"

Internally, a Hashtable is an **Array of Entry Objects**. Each entry is essentially a key-value pair.

The Step-by-Step Process:

1. **Hashing:** When you call put(key, value), the Hashtable calls the key's hashCode() method to get a raw integer.
2. **Indexing:** To fit that hash into the array, it uses the formula:

$$\text{\$index} = (\text{hash} \ \& \ 0x7FFFFFFF) \ \% \ \text{arrayLength}\$$$
3. **Collision Handling:** If two keys land on the same index (a Collision), Hashtable uses **Separate Chaining**. It stores the entries in a **Singly Linked List** at that array index.
4. **Retrieval:** When you call get(key), it calculates the index, goes to the bucket, and traverses the linked list using .equals() to find the exact match.

2. Why, How, When, and Where?

WHY use Hashtable?

The only reason to use Hashtable is for **Thread Safety**. Every single method in Hashtable (like put, get, remove) is marked with the synchronized keyword. This ensures that only one thread can modify or read from the table at a time.

HOW does it differ from HashMap?

This is the "Golden Question" for placements:

Feature	Hashtable	HashMap
Synchronization	Synchronized (Thread-safe)	Non-synchronized (Not thread-safe)
Null Keys/Values	Strictly Forbidden (Throws NullPointerException)	Allows one null key and multiple null values
Performance	Slow (due to locking overhead)	Fast
Iteration	Uses Enumerator and Iterator	Uses only Iterator
Legacy	Part of original JDK 1.0	Part of Collections Framework (JDK 1.2)

WHEN to use it?

- **Short Answer:** Almost **never** in modern Java.
- **Long Answer:** Use it only if you are maintaining very old legacy code (pre-Java 5).

WHERE is it replaced?

- If you **don't** need thread safety: Use **HashMap**.
- If you **do** need thread safety: Use **ConcurrentHashMap**.

Why? Hashtable locks the **entire table**, making it slow. ConcurrentHashMap locks only **segments** (buckets), allowing multiple threads to work simultaneously.

3. The "Fail-Safe" vs "Fail-Fast" Nuance

Interviewers use this to see if you really know the internals:

- **HashMap's Iterator is Fail-Fast:** If you modify the map while iterating, it throws ConcurrentModificationException.
- **Hashtable's Enumerator is Fail-Safe:** It doesn't throw an exception if the table is modified during iteration (it works on a copy or a specific state), but this is generally considered less predictable.

4. Key Performance Factor: Load Factor

Hashtable has a default **Initial Capacity of 11** and a **Load Factor of 0.75**.

- When the table is 75% full, it performs a **Rehash**. It creates a new array (usually double the size + 1) and moves all existing elements to new positions. This is an expensive $O(n)$ operation.

ConcurrentHashMap

If Hashtable is the "Legacy Sibling" and HashMap is the "Fast Sibling," then **ConcurrentHashMap** is the "Professional Sibling."

It was introduced in **Java 5** to solve the performance bottlenecks of Hashtable while maintaining total thread safety. In modern high-frequency trading or web server development, this is the most used Map implementation.

1. The "Why": The Problem with Hashtable

In Hashtable, the entire map is synchronized. If Thread A is reading a value from Bucket 1, Thread B is **blocked** from even looking at Bucket 10. This is called **Global Locking**. It creates a massive "traffic jam" in multi-threaded apps.

ConcurrentHashMap solves this by using **Lock Striping** or **Bucket-Level Locking**.

2. Internal Working (Evolution)

Pre-Java 8: Segmented Locking

The map was divided into **Segments** (default 16). Each segment acted like an independent Hashtable with its own lock.

- **Result:** 16 threads could write to the map at the same time, as long as they were hitting different segments.

Java 8 and Beyond: CAS and Synchronized Buckets

Java 8 completely redesigned it for even higher performance:

1. **CAS (Compare-And-Swap):** When inserting into an empty bucket, it uses a "wait-free" CPU instruction called CAS. No locks are used!
2. **Bucket-Level Locking:** If a collision occurs (the bucket is not empty), it synchronizes **only that specific bucket's head node**.
3. **Tree-fying:** Just like HashMap, if a bucket gets too crowded (8+ elements), it converts the Linked List into a **Red-Black Tree** to keep search time at $O(\log n)$.

3. Key Characteristics

Feature	ConcurrentHashMap
Thread Safety	Fully thread-safe without locking the whole map.
Null Keys/Values	Strictly Prohibited (like Hashtable).
Iterators	Fail-Safe (does not throw ConcurrentModificationException). It reflects the state of the map at the time the iterator was created.
Performance	Extremely high in multi-threaded environments.
Locking Type	Fine-grained (Bucket-level) locking.

4. When to use what? (The "Interview Verdict")

- **Single-threaded:** Use HashMap.
- **Multi-threaded (Low concurrency):** You could use Collections.synchronizedMap(hashMap), but it's basically as slow as Hashtable.
- **Multi-threaded (High concurrency/Production):** Always use ConcurrentHashMap.

5. Why are Nulls forbidden in ConcurrentHashMap?

This is a high-level "brain teaser" question.

- **The Reason:** In a multi-threaded environment, if get(key) returns null, you cannot distinguish between:
 1. The key is not present in the map.
 2. The key is present, but its value is literally null.
- In a single-threaded HashMap, you can check containsKey(). But in ConcurrentHashMap, the map might change between your call to get() and containsKey(). To avoid this ambiguity (the "Liveness" problem), nulls are banned.

6. Common Interview Scenario

Interviewer: "I have a global cache shared by 100 threads. Which Map should I use?"

Your Answer: "I will use ConcurrentHashMap. It allows multiple threads to read concurrently without any locking and allows multiple threads to write into different buckets simultaneously. This provides much higher throughput than Hashtable or synchronizedMap."

1. The Structure: Node Array

At its heart, ConcurrentHashMap uses an array of **Nodes**.

- Each Node contains the hash, key, value, and a reference to the next node (for chaining).
- The value and next pointers are marked as **volatile**. This is critical: it ensures that when one thread updates a value, all other threads see that change immediately without needing a lock for reading.

2. The "No-Lock" Insertion (CAS)

When you call put(key, value), the map first calculates the hash and determines the bucket index. If the bucket is **empty**, Java does not use a lock.

- It uses **CAS (Compare-And-Swap)** operations via the Unsafe class.
- It says: *"If the bucket is still null, put this new node here."* * If another thread snuck in and filled the bucket a millisecond before, the CAS fails, and the thread simply retries the logic. This "optimistic" approach makes insertions into empty buckets incredibly fast.

3. The "Fine-Grained" Lock (Synchronized)

If the bucket is **not empty** (a collision), the map needs to be more careful.

- It uses the synchronized keyword, but **only on the first node** (the head) of that specific bucket.
- **Why this is brilliant:** Thread A can be updating a Linked List in Bucket 1 while Thread B is updating a Red-Black Tree in Bucket 5. They do not block each other. Only two threads trying to hit the **exact same bucket** will face a lock.

4. The "Tree-fication" Logic

Just like a regular HashMap, if a bucket's Linked List grows beyond a threshold (8 elements) and the total map capacity is at least 64:

- The bucket is converted into a **TreeBin** (Red-Black Tree).
- This ensures that even under a "Hash Attack" or poor hashing, the search time stays at $O(\log n)$ instead of $O(n)$.

5. Concurrent Resizing (Transferring)

This is the most complex part of the internal works. In a regular HashMap, resizing is a "stop-the-world" event. In ConcurrentHashMap, resizing is **collaborative**.

- When the load factor is exceeded, a ForwardingNode is placed in the buckets.
- If a thread tries to put an element and sees a ForwardingNode, it actually **helps** move other buckets to the new, larger array before finishing its own insertion.
- This "multithreaded resizing" prevents a single thread from getting stuck doing all the heavy lifting.

6. How "Get" works without Locks

The get() method is **completely lock-free**.

1. It calculates the hash and finds the bucket.
2. If the first node matches, it returns the value.
3. Because the value field is volatile, the thread is guaranteed to see the most recent write.
4. If it's a ForwardingNode, it follows the reference to the new table.
5. If it's a TreeBin, it performs a tree search.

7. The "Size" Problem

How do you count elements when 100 threads are adding them simultaneously?

- ConcurrentHashMap doesn't use a single volatile int size. That would create a bottleneck as every thread fights to update that one counter.
- Instead, it uses a **CounterCell array** (similar to LongAdder). Threads update different cells in the array, and only when you call size(), the map sums them all up.

LinkedHashMap

If HashMap is about **speed** and TreeMap is about **sorting**, then LinkedHashMap is about **order**.

In a standard HashMap, the order in which you insert elements is not preserved. If you put "A", then "B", then "C", a HashMap might print them as "B", "C", "A". A LinkedHashMap fixes this by remembering the sequence.

1. Internal Working: The "Hybrid" Structure

LinkedHashMap extends HashMap. It uses the same hash table mechanics (buckets, nodes, and arrays) but adds a **Doubly Linked List** running through all its entries.

- **The Hash Map Part:** Handles the storage, hashing, and search performance.
- **The Linked List Part:** Connects every entry in the map to the one inserted before it and after it.

Each node in a LinkedHashMap is a LinkedHashMap.Entry, which looks like this:

Java

```
static class Entry<K,V> extends HashMap.Node<K,V> {
    Entry<K,V> before, after; // Pointers for the Doubly Linked List
    // Inherits key, value, hash, and next from HashMap.Node
}
```

2. Why, How, When, and Where?

WHY use it?

To maintain a predictable iteration order. In many applications (like a Recent Search history or a Navigation menu), the order of items matters as much as the items themselves.

HOW does it maintain order?

It has two modes, controlled by a boolean flag in the constructor (accessOrder):

1. **Insertion Order (Default):** The iteration order is exactly how you put the elements in.
2. **Access Order:** Every time you get() or put() an element, it moves to the **end** of the list. This makes it a "Most Recently Used" (MRU) structure.

WHEN to use it?

- When you need a map that behaves like a HashMap but keeps things in order.
- When building a **Cache** (specifically an LRU Cache).

WHERE is it better than TreeMap?

- TreeMap sorts by keys (A-Z or 1-10). It takes time.
- LinkedHashMap maintains order by insertion/access. It takes time.
- If you don't need things sorted alphabetically but just want them in the order they arrived, LinkedHashMap is significantly faster.

3. The "LRU Cache" (The Interviewer's Secret)

This is the #1 reason LinkedHashMap is asked in interviews. You can create a **Least Recently Used (LRU) Cache** in just a few lines of code by overriding one specific method: removeEldestEntry.

Java

```
LinkedHashMap<Integer, String> lruCache = new LinkedHashMap<>(16, 0.75f, true) {
    @Override
    protected boolean removeEldestEntry(Map.Entry eldest) {
        return size() > 5; // Automatically removes the oldest item if size exceeds 5
    }
};
```

When accessOrder is true, the "eldest" entry is the one that hasn't been touched for the longest time.

4. Comparison Summary

Feature	HashMap	LinkedHashMap	TreeMap
Iteration Order	Random / Unpredictable	Insertion or Access Order	Sorted Order (Natural or Comparator)
Search Time			
Internal Logic	Array + Linked List/Tree	Array + Doubly Linked List	Red-Black Tree
Null Keys	Allowed	Allowed	Not Allowed (throws NullPointerException)

5. Potential Interview Questions

Q1: Does LinkedHashMap consume more memory than HashMap?

- **Answer:** Yes. Each entry has to store two additional pointers (before and after). For very large maps, this overhead can be significant.

Q2: Is LinkedHashMap thread-safe?

- **Answer:** No. Like HashMap, it is not synchronized. You must use Collections.synchronizedMap() or handle locking manually if multiple threads are involved.

Q3: What happens if I re-insert an existing key?

- **Answer:** In **Insertion Order** mode, the position does not change; only the value is updated. In **Access Order** mode, the entry moves to the very end of the list because it was "accessed."

TREEMAP

If HashMap is a **storage bin** and LinkedHashMap is a **queue**, then **TreeMap** is a **sorted filing cabinet**.

It is the go-to implementation when you need your keys to be automatically sorted in a specific order (like alphabetical or numerical) at all times.

1. Internal Working: The Red-Black Tree

Unlike HashMap, which uses hashing and arrays, TreeMap is based on a **Red-Black Tree** (a type of self-balancing Binary Search Tree).

- **Nodes:** Each entry is a node containing a Key, Value, and pointers to its left, right, and parent nodes, along with a color (Red or Black).
- **Sorting:** When you add an entry, it is placed in the tree based on the key's value.
- **Self-Balancing:** To prevent the tree from becoming a long, thin line (which would make it slow), it "rotates" and changes colors whenever you insert or delete something. This keeps the tree height minimal.
- **Time Complexity:** Because it is a balanced tree, operations like put, get, and remove take $O(\log n)$ time.

2. Why, How, When, and Where?

WHY use it?

Use TreeMap when you need to perform **range-based operations**. Because it is sorted, it can quickly tell you:

- What is the smallest or largest key? (firstKey(), lastKey())
- What are all the keys between "A" and "F"? (subMap())
- What is the closest key greater than "M"? (ceilingKey())

HOW does it sort?

It uses one of two mechanisms:

1. **Natural Ordering:** Uses the Comparable interface (e.g., Strings are A-Z, Integers are 1-10).
2. **Custom Ordering:** You can provide a Comparator in the constructor to define your own rules (e.g., sorting Strings by their length).

WHEN to use it?

- When you need a map that is **always sorted**.
- When you need to frequently find "floor," "ceiling," "head," or "tail" of a dataset.
- **Placement Tip:** If the interview question involves "finding the top 10 elements" or "sorting a dictionary," think TreeMap.

WHERE is it restricted?

- **Null Keys:** TreeMap **does not allow null keys**. Since it needs to compare the new key with existing keys to know where to put it, calling null.compareTo() would throw a NullPointerException. (Note: Null *values* are allowed).

3. Comparison with HashMap

Feature	HashMap	TreeMap
Search Time	$O(1)$ (Constant)	$O(\log n)$ (Logarithmic)
Ordering	No Order	Sorted Order
Internal Works	Hash Table	Red-Black Tree
Null Keys	Allowed (One)	Not Allowed
Memory	Lower	Higher (due to pointers and balancing)

4. Key Methods to Remember (The "Navigable" part)

TreeMap implements the NavigableMap interface, which gives you these "superpowers":

- higherKey(K key): Returns the smallest key strictly greater than the given key.
- descendingMap(): Returns a reverse-order view of the map.
- pollFirstEntry(): Removes and returns the entry with the smallest key.

5. Potential Interview Questions

Q1: What happens if you try to put a key that does not implement Comparable?

- **Answer:** It will throw a ClassCastException at runtime unless you provided a custom Comparator when creating the TreeMap.

Q2: Which is faster for insertion, HashMap or TreeMap?

- **Answer:** HashMap is generally faster ($O(1)$ vs $O(\log n)$). However, if the hash function is poor and causes many collisions, HashMap performance can degrade, whereas TreeMap remains consistently $O(\log n)$.

Q3: Can we sort a TreeMap by Values?

- **Answer:** No, TreeMap only sorts by **Keys**. If you need to sort by values, you would typically convert the map to a List and use Collections.sort() with a custom comparator.

Can HashMap have null keys?

The short answer is **yes**, a HashMap can have **one null key** and multiple null values.

However, to truly ace a placement interview, you need to understand *how* it handles that null key internally, especially since other maps like Hashtable and TreeMap do not allow it.

1. How it works internally

Normally, HashMap calculates the bucket index using the key's hashCode():

$\text{index} = \text{hash}(\text{key}) \& (n-1)$

The Problem: If you try to call .hashCode() on a null reference, Java will throw a NullPointerException.

The Solution: The HashMap source code has a specific "null-check" in its hash() method. If the key is null, it **explicitly returns a hash value of 0**.

Java

```
static final int hash(Object key) {
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
```

Because the hash is always 0 for a null key, **the null key is always stored in the very first bucket (index 0) of the internal array**.

2. Why only ONE null key?

A HashMap cannot have duplicate keys. When you put a second null key, the HashMap goes to index 0, finds the existing null entry, and simply **overwrites** the old value with the new one.

3. Comparison with other Maps

Interviewers love to compare this behavior across the "Map Family":

Map Type	Allows Null Key?	Why / Why Not?
HashMap	Yes (One)	Explicitly handled to return hash 0.
LinkedHashMap	Yes (One)	Inherits HashMap logic.
Hashtable	No	Throws NullPointerException (Legacy design).
TreeMap	No	Needs to call compareTo(); null cannot be compared.
ConcurrentHashMap	No	To avoid ambiguity in multi-threaded "get" vs "contains" checks.

4. The "Ambiguity" Trap (Interview Scenario)

Interviewer: "If map.get(key) returns null, does it mean the key doesn't exist?"

Your Answer: "Not necessarily in a HashMap. It could mean the key doesn't exist, **OR** it could mean the key exists and its value is explicitly set to null. To be sure, I should use map.containsKey(key)."

Summary for Placements:

- HashMap allows **one** null key.
- It is always stored at **index 0**.
- It handles it by skipping the .hashCode() call and forcing the hash to be 0.

1. What is a Collision?

In a HashMap or Hashtable, a **Collision** occurs when two different keys produce the same array index.

Even if key1.equals(key2) is **false**, their hashCode() values (after the internal hash function is applied) might point to the same bucket in the underlying array.

How Java 8+ Handles It:

1. **Chaining:** Initially, Java stores the colliding elements in a **Singly Linked List** at that bucket.
2. **Tree-ification:** If a single bucket gets too crowded (8 or more elements), the Linked List is converted into a **Red-Black Tree**. This improves search time from $O(n)$ to $O(\log n)$.

2. What is Load Factor?

The **Load Factor** is a measure that decides when the HashMap should increase its capacity (resize).

- **Formula:**

- **The Default:** 0.75.
- **The Logic:** When the map becomes 75% full, it doubles its size. This prevents the buckets from getting too long, which would slow down your search time.

3. Why is the Default Size 16?

The initial capacity of a HashMap is 16, and it must always be a **power of 2** (16, 32, 64, etc.).

The Mathematical Secret:

Java determines the bucket index using the formula:

Where is the capacity.

- If is a power of 2 (like 16), then is a sequence of all 1s in binary (like 1111).
- Doing a bitwise AND (&) with a sequence of 1s ensures that the hash bits are distributed **uniformly** across all buckets. This minimizes collisions. If the size were an odd number, some buckets might never be used.

4. Why is the Load Factor 0.75?

The value 0.75 was chosen after extensive mathematical research (based on **Poisson Distribution**) as a "Sweet Spot" between **Time** and **Space**.

Load Factor	Space (Memory)	Time (Speed)
Low (0.5)	Wastes memory (resizes too often).	Very fast (fewer collisions).
High (1.0)	Saves memory.	Very slow (too many collisions/long chains).
Default (0.75)	Optimal.	Optimal.

Statistical analysis shows that with a load factor of 0.75, the probability of a bucket having more than 8 elements is roughly 1 in 10 million. It's the perfect compromise.

5. How does an Object "Die" in Java?

In languages like C++, you must manually "kill" an object. In Java, objects die when they are no longer **reachable**. This process is managed by the **Garbage Collector (GC)**.

The Stages of Death:

1. **Unreachability:** An object is eligible for death when no "Live Threads" have a reference to it.
2. **Marking:** The GC traverses the "Object Tree" starting from "GC Roots" (like static variables or local variables on the stack). Any object it cannot reach is marked for deletion.
3. **Finalization (Optional):** If the object overrides the finalize() method, it is added to a finalization queue. (Note: finalize is deprecated; don't use it!).
4. **Sweeping:** The GC reclaims the memory used by the marked objects and makes it available for new objects.
5. **Compaction:** To prevent memory fragmentation (holes in memory), the GC moves the surviving objects together, leaving a single large block of free space.

The "Generational" Death Row:

Objects don't all die in the same place. Most objects die young in the **Eden Space** (Young Generation). If they survive several GC cycles, they are promoted to the **Old Generation**, where they live longer before being collected.

5-Why use arraylist linked list and vector in java?

1. ArrayList (The Go-To Choice)-ArrayList is backed by a Dynamic Array. When it runs out of space, it creates a new, larger array and copies the elements over.

- **Best for:** Random access and search operations.
- **Why use it?** It provides $O(1)$ time complexity for retrieving an element using an index (get(index)).
- **Drawback:** Inserting or deleting elements in the middle of the list is slow ($O(n)$) because all subsequent elements must be shifted.

2. LinkedList (The Specialist)-LinkedList is backed by a Doubly Linked List. Each element (node) stores the actual data and pointers to the previous and next nodes.

Feature	ArrayList	LinkedList	Vector
Data Structure	Dynamic Array	Doubly Linked List	Dynamic Array
Random Access	$O(1)$ (Fastest)	$O(n)$ (Slow)	$O(1)$
Insertion/Deletion	$O(n)$ (Shifting required)	$O(1)$ (No shifting)	$O(n)$
Thread Safety	No	No	Yes
Memory Usage	Low	High (due to pointers)	Low

- **Best for:** Frequent insertions and deletions, especially at the beginning or end.
- **Why use it?** Adding or removing an element is very fast ($O(1)$) because you only need to update the pointers of the neighboring nodes—no shifting is required.
- **Drawback:** Random access is slow ($O(n)$) because to find the 50^{th} element, the JVM must start at the beginning and follow the pointers one by one. It also consumes more memory due to the overhead of storing two pointers per node.

3. Vector (The Legacy Option)-Vector is very similar to ArrayList (it also uses a dynamic array), but it is Synchronized.

- **Best for:** Thread-safe environments (though modern alternatives are usually better).
- **Why use it?** It is "thread-safe," meaning multiple threads can access it without corrupting the data.
- **Drawback:** Synchronization adds significant overhead, making it much slower than ArrayList. In modern Java, developers usually prefer ArrayList with manual synchronization or CopyOnWriteArrayList for better performance.

1-What are objects and classes? Where are they used in real systems?

- **The Class (The Blueprint)**-A **class** is a logical template or a blueprint. It defines a set of attributes (data) and behaviors (functions/methods) that all objects of that type will have. It doesn't occupy memory for data itself; it just describes what the data will look like
- **The Object (The Instance)**-An **object** is a self-contained unit created from a class. It is a "real" entity that occupies memory and holds actual values. You can create multiple objects from a single class, each with its own unique data.
- **Example:** Class is architectural blueprint of a house (dimensions, number of rooms, where the plumbing goes). Object is actual physical house built on a specific street. You can build ten houses from one blueprint; they all have the same structure, but one might be painted blue while another is green.
- **Attributes (States):** What the object *knows* (e.g., color, price, name).
- **Methods (Behaviors):** What the object *does* (e.g., drive, stop, calculate).

Feature	Class	Object
Definition	A blueprint or template for creating objects.	An instance of a class.
Nature	It is a logical entity .	It is a physical entity .
Memory	Does not consume memory when defined.	Consumes memory as soon as it is created.
Declaration	Declared once in the code.	Can be created many times (as many as needed).

Banking Systems:

- ✓ **Class:** BankAccount
- ✓ **Attributes:** accountNumber, accountHolderName, balance.
- ✓ **Methods:** deposit(), withdraw(), checkBalance().
- ✓ **Objects:** Your personal savings account and your friend's checking account are separate objects. When you withdraw money, it only affects the balance attribute of *your* object.

2-What are the 4 pillars of OOP? Why are they important?

1. Abstraction-It is the practice of hiding internal complexity and showing only the essential features of an object. It focuses on *what* an object does rather than *how* it does it.

- **Real-World Example:** When you use a TV remote, you press "Power" to turn it on. You don't need to understand the internal circuitry or how the infrared signal is processed to use the device.
- **Importance:** It reduces complexity for the user (or other parts of the code) by providing a simple interface.
 - **A. Reduced Complexity**-Modern systems are too large for one person to understand every line of code. Abstraction allows a developer to understand just a few commands, without needing to know the thousands of lines of code inside it.
 - **B. Easy Maintenance and Updates**-If you hide the implementation, you can change it later without affecting the people using it.**Example:** If a banking app decides to switch its backend database from MySQL to MongoDB, but the "Withdraw" function looks the same to the user, the change is "invisible." If the details weren't hidden, every part of the app that touches the database would break.

- **C. Increased Security and Integrity**-By hiding the implementation, you prevent users from bypassing safety checks. If a user only has access to a `sendMessage()` method, they can't accidentally (or intentionally) modify the underlying network protocols or packet headers that could crash the system.
- **D. Code Reusability**-Abstraction allows you to create generic templates. For example, you can define a general `Shape` class with an abstract method `calculateArea()`. Whether the specific shape is a `Circle` or a `Square`, the rest of your program just needs to know that it *can* calculate an area, making the code much more flexible.

Feature	Abstraction	Encapsulation
Primary Goal	To hide unnecessary implementation details so the user sees only the relevant features.	To bundle data and methods together and restrict access to prevent unauthorized tampering.
Focus	Focuses on Design (What the object does).	Focuses on Implementation (How the object does it).
Mechanism	Achieved using Abstract Classes and Interfaces .	Achieved using Access Modifiers (<code>private</code> , <code>protected</code> , <code>public</code>).
Example	Using a TV remote: you only care about the "Power" button, not the circuitry inside.	A "Bank Account" class where the balance is <code>private</code> and can only be modified through a <code>deposit()</code> method.

2. Encapsulation-It is the bundling of data (attributes) and the methods that operate on that data into a single unit (a class).

Crucially, it involves **data hiding**—restricting direct access to some of an object's components to prevent accidental interference.

- **Real-World Example:** A medical capsule. The medicine is inside, protected by the outer shell. You interact with the pill as a single unit, but you cannot change the chemical powder inside directly.
- **Importance:** It keeps data safe from outside corruption and makes the code easier to maintain, as you can change the internal logic without breaking everything else.
- it means making the fields in a class `private` and providing access to them via public methods (commonly called getters and setters).
- **Data Hiding** ensures that the internal details of a class are hidden from outside classes. By using access modifiers like `private`, you prevent external code from reaching in and changing a variable directly.
 - **A. Preventing "Impossible" States (Validation)**-If you have a `BankAccount` class with a `balance` variable, you don't want other parts of the program to set `balance = -1000000` directly. Without Hiding: Any developer could accidentally set a negative balance. With Hiding: The variable is `private`. To change it, you must use a `deposit()` or `withdraw()` method that includes logic to check if the transaction is valid.
 - **B. Flexibility and Maintenance**-If you decide to change how data is stored (for example, switching a temperature variable from Fahrenheit to Celsius), you only have to update the code inside the class.
 - **C. Complexity Management**-A large system has thousands of variables. Data hiding ensures that a programmer only sees the parts of an object they *need* to interact with, reducing mental load and the chance of bugs.
 - **D. Security**-In sensitive systems, data hiding ensures that unauthorized parts of the code cannot view or modify critical information. It enforces a "need-to-know" basis for software components.

3. Inheritance-It allows a new class (subclass) to acquire the properties and behaviors of an existing class (superclass). This creates a hierarchy.

- **Real-World Example:** In a biological hierarchy, a "Dog" inherits traits from the "Mammal" group. A Dog *is a* Mammal, so it automatically has features like being warm-blooded, but it also has its own specific traits like barking.
- **Importance:** It promotes **code reusability**. You don't have to rewrite common logic for every new object; you just inherit it. **Scalability and Maintenance**-If you need to add a new feature that applies to all employees (like a `calculateTax()` method), you only have to add it to the parent class.

4. Polymorphism-It (meaning "many forms") allows different classes to be treated as instances of the same general class through the same interface. The most common types are **Method Overloading** (static) and **Method Overriding** (dynamic).

- **Real-World Example:** Think of the command "Speak." A Dog object will respond by barking, a Cat object will meow, and a Bird object will chirp. The command is the same, but the result depends on the object receiving it.
- **Importance:** It allows for flexibility and "plug-and-play" code. You can write a function that handles any "Animal" without needing to know exactly which type it is until the program runs. Code Cleanliness (Reduced Logic Branches).

- ✓ **DRY (Don't Repeat Yourself):** Inheritance and Abstraction ensure you write logic once and reuse it everywhere.
- ✓ **Security:** Encapsulation ensures that sensitive data is only modified in controlled ways, preventing bugs.

- ✓ **Scalability:** As a system grows (like a massive hospital management system or an e-commerce platform), these pillars keep the code modular so teams can work on different parts without stepping on each other's toes.

✓ **Flexibility:** Polymorphism allows you to add new features (like a new type of payment method) with minimal changes to the existing codebase.

3-Types of Polymorphism.

A. Compile-Time Polymorphism (Method Overloading)-

This occurs when multiple methods in the same class have the same name but different parameters (different number or types of arguments). The compiler determines which method to call at compile time. It makes code readable and user-friendly.

- **Key Rule:** The method signature must change.

Feature	Method Overloading	Method Overriding
Location	Within the same class .	Between Parent and Child classes.
Parameters	Must be different .	Must be the same .
Binding	Static / Compile-time.	Dynamic / Run-time.
Purpose	To provide multiple ways to do the same task.	To provide a specific version of a generic task.

B. Runtime Polymorphism (Method Overriding)-This

occurs when a subclass provides a specific implementation for a method that is already defined in its parent class. The decision of which method to run is made while the program is actually running. Overriding allows system to be scalable.

- **Key Rule:** The method signature (name and parameters) must be exactly the same.

🌈 When you call a method, the compiler matches the arguments you provided to the corresponding method signature. If it finds a match, it binds that specific method call to that block of code. This is called **Static Binding**.

Feature	Static Binding	Dynamic Binding
When?	During Compilation (Before the app opens).	During Runtime (While the app is open).
Speed	Very fast.	Slightly slower (tiny lookup delay).
Flexibility	Rigid (Fixed logic).	Highly flexible (Adapts to objects).
Example	Method Overloading.	Method Overriding.

3-JAVA 7 VS JAVA 8.

Major Features Introduced in Java 8:

1-Lambda Expressions-Before Java 8, if you wanted to pass a piece of logic, you had to use Anonymous Inner Classes. Java 8 introduced Lambdas to treat "code as data," making the syntax much more concise.**Java 7:** new Runnable() { public void run() { ... }}

Java 8: () -> { ... }

2-Functional Interfaces-Java 8 introduced the @FunctionalInterface annotation. These are interfaces with exactly one abstract method (e.g., Predicate, Function, Consumer).

3-Streams API-This allowed for declarative data processing (filtering, mapping, sorting) on collections, similar to SQL queries. It also made parallel processing significantly easier.

4-Optional Class-A new container object used to represent the presence or absence of a value. It was designed to reduce the frequency of the dreaded NullPointerException.

5-Metaspace replacing PermGen-In Java 7, the PermGen (where class metadata lived) had a fixed size, often leading to java.lang.OutOfMemoryError: PermGen space. Java 8 replaced it with Metaspace, which uses native memory and expands automatically.

3. Performance & Optimization-HashMap Improvement: Java 8 introduced Treeification. When a bucket in a HashMap exceeds a certain threshold (8 elements), the linked list is converted into a Red-Black Tree, improving the worst-case performance from O(n) to O(log n).**Parallel Array Sorting:** The Arrays.parallelSort() method was added to utilize multi-core processors for faster sorting of large datasets.

4-Custom Exceptions?

To create one, you simply define a new class that extends an existing exception class from the Java hierarchy.

Step A: Choose the Parent Class

- **Checked Exception:** Extend the Exception class. The compiler will force the caller to handle this exception (using try-catch or throws).
- **Unchecked Exception:** Extend the RuntimeException class. This is usually preferred for programming errors or business logic failures where you don't want to clutter the code with mandatory try-catch blocks.

Step B: Define the Class

Feature	Java 7	Java 8
Programming Paradigm	Imperative / Object-Oriented	Functional / Object-Oriented
Memory Management	Permanent Generation (PermGen)	Metaspace (Native Memory)
Interface Logic	Methods could only be abstract	Can have Default and Static methods
Iterators	External (For-each, Iterator)	Internal (Streams API)
Date & Time	<code>java.util.Date</code> (Not thread-safe)	<code>java.time</code> (Immutable & Thread-safe)

Standard practice is to provide at least two constructors: one that accepts a message and one that accepts both a message and the original cause (for exception chaining).

```
// Custom Unchecked Exception
public class InvalidAgeException extends RuntimeException {

    // Constructor that accepts a custom message
    public InvalidAgeException(String message) {
        super(message);
    }

    // Constructor that accepts a message and a cause
    public InvalidAgeException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

```
public class Election {
    public void checkEligibility(int age) {
        if (age < 18) {
            // Triggering the custom exception
            throw new InvalidAgeException("Age " + age + " is not eligible to vote");
        }
        System.out.println("Welcome to the election!");
    }
}
```

Step C: Use the Exception

You use the throw keyword to trigger the exception when a specific condition is met.

❖ **Can a custom exception contain extra fields, like an 'errorCode'?** Yes! This is a major advantage of custom exceptions—you can store a numeric code or a timestamp to help with logging and debugging.

Technology	Latest Stable/Innovation	Latest LTS (Recommended)
Java	26	25
MySQL	9.6.0	8.4.8